idat

# Modelos de Calidad de Software

Escuela de Tecnología

**Mg.** Daniel Arias, **PMP™** y **Scaled Scrum Master™**

# ¿Qué aprendimos la clase pasada?

- SCRUM
- Ciclo de vida SCRUM
- Roles y ceremonias
- Historia de usuarios
- Creación de casos de prueba basados en historias de usuario.
- Que es Cucumber y gherkin



idat

# Conversemos

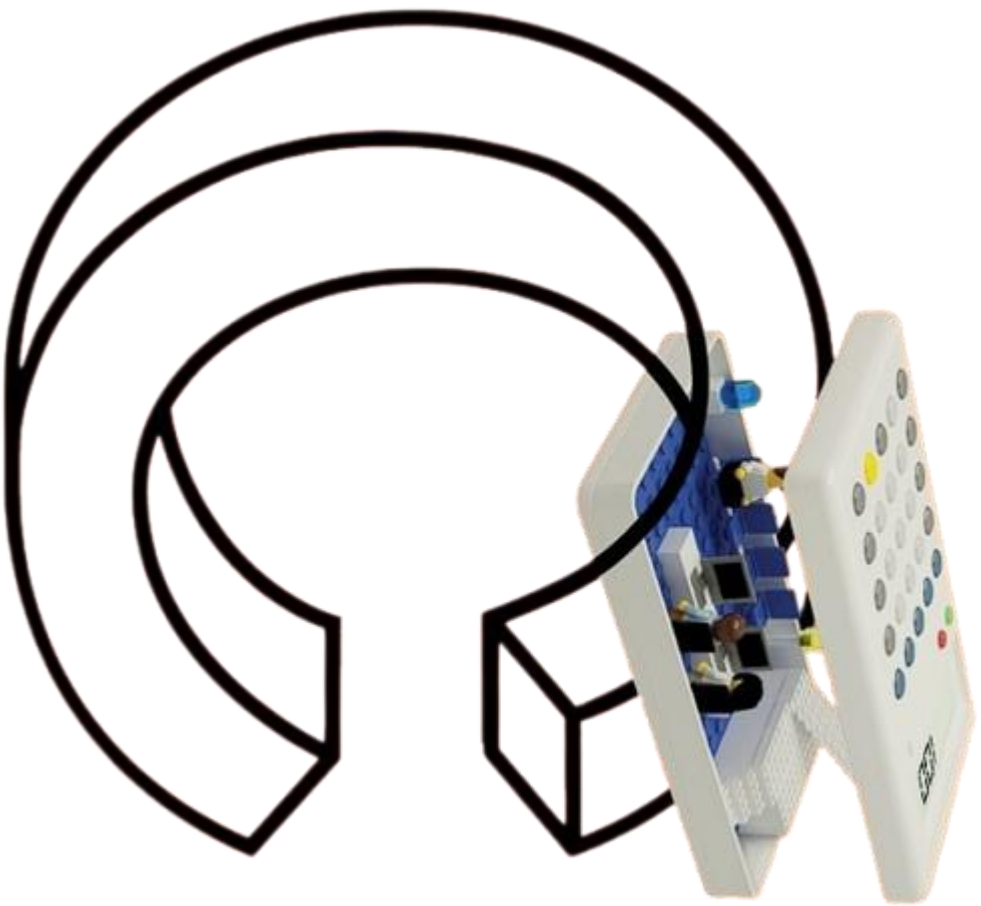



http://www.menti.com

idat

# Introducción a pruebas agiles – Parte 2

Semana 13



## Logro de aprendizaje

- Creación de proyecto java con cucumber
- Estructura de un archivo feature
- Automatización básica empleando cucumber y java
- Automatización básica empleando cucumber y selenium
- Introducción a JMeter

## Actividad de aprendizaje

ACTIVIDAD VIRTUAL

Revisa y analiza el enlace sugerido en la plataforma virtual luego realiza la actividad propuesta

- Efectuar la automatización de las historias de usuarios creadas empleando cucumber

idat

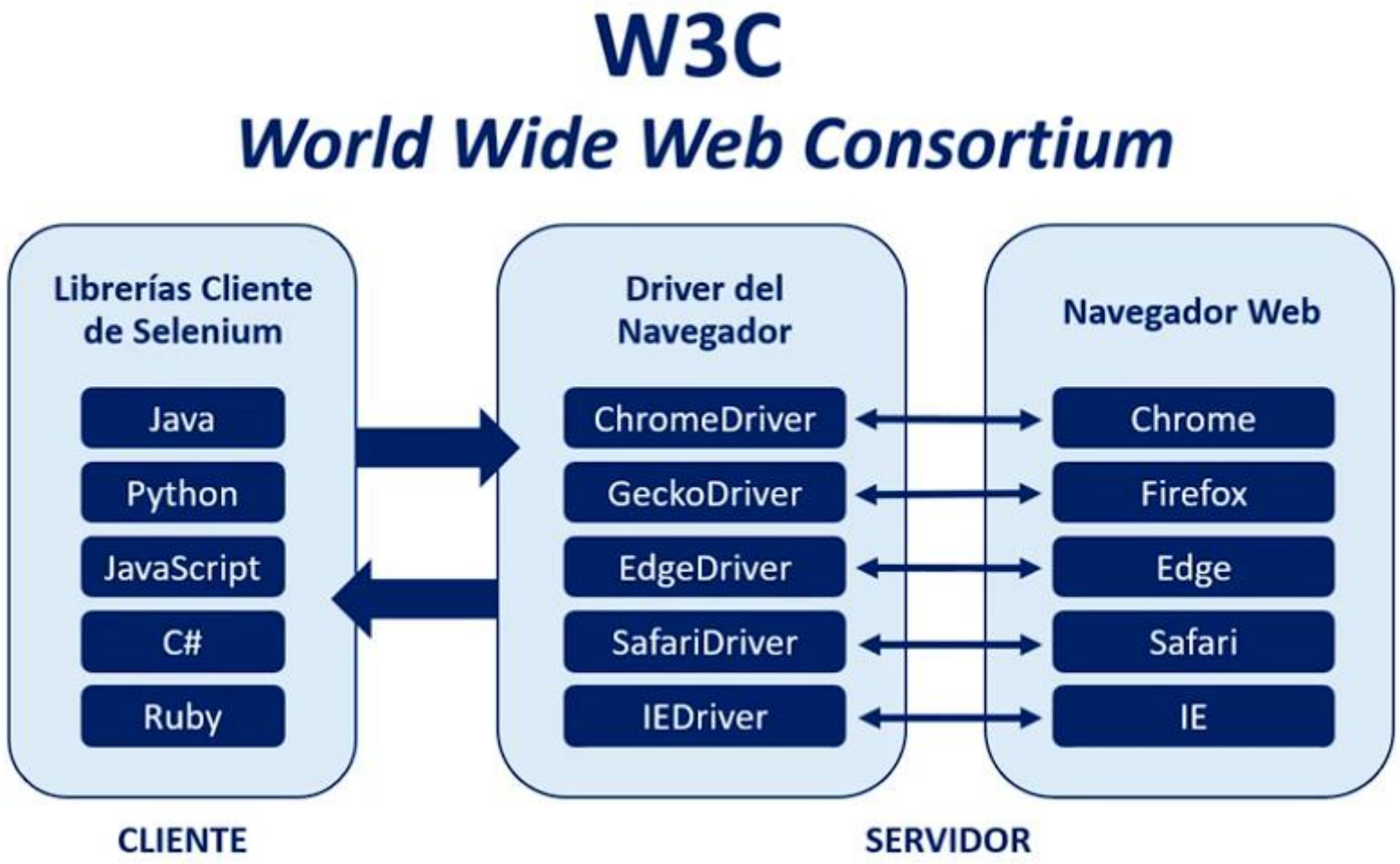
## Logro de aprendizaje

Automatizar los escenarios, usando cucumber.



idat

# Arquitectura de Selenium

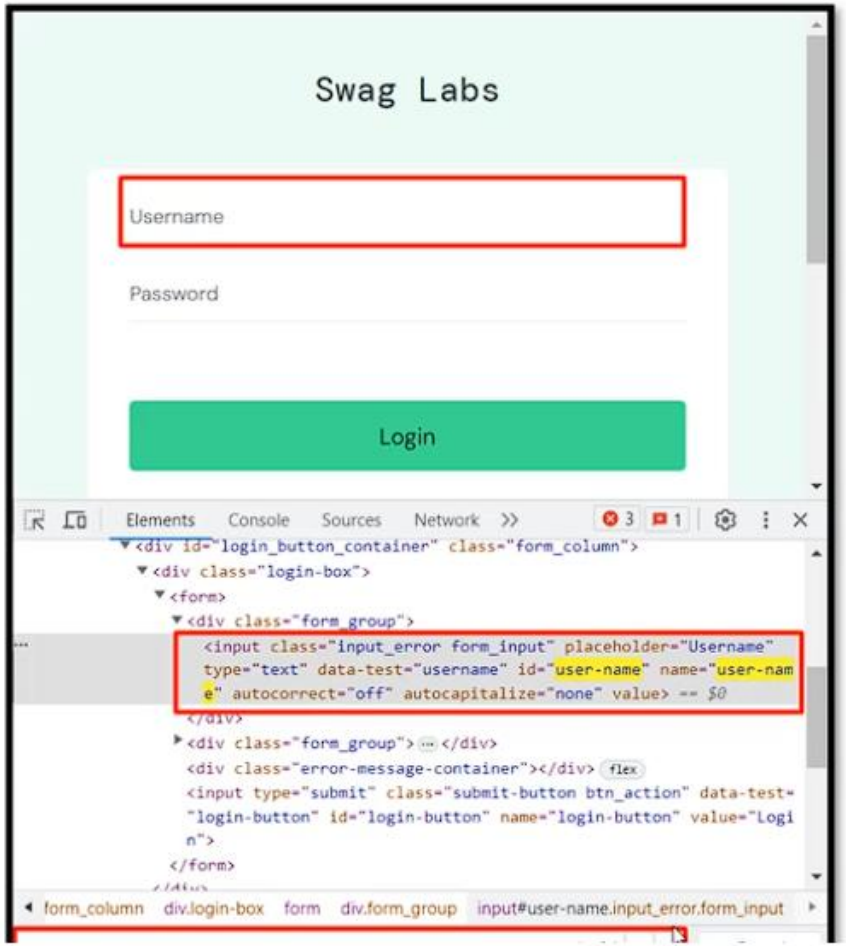



idat

# Tipos de localizadores en Selenium

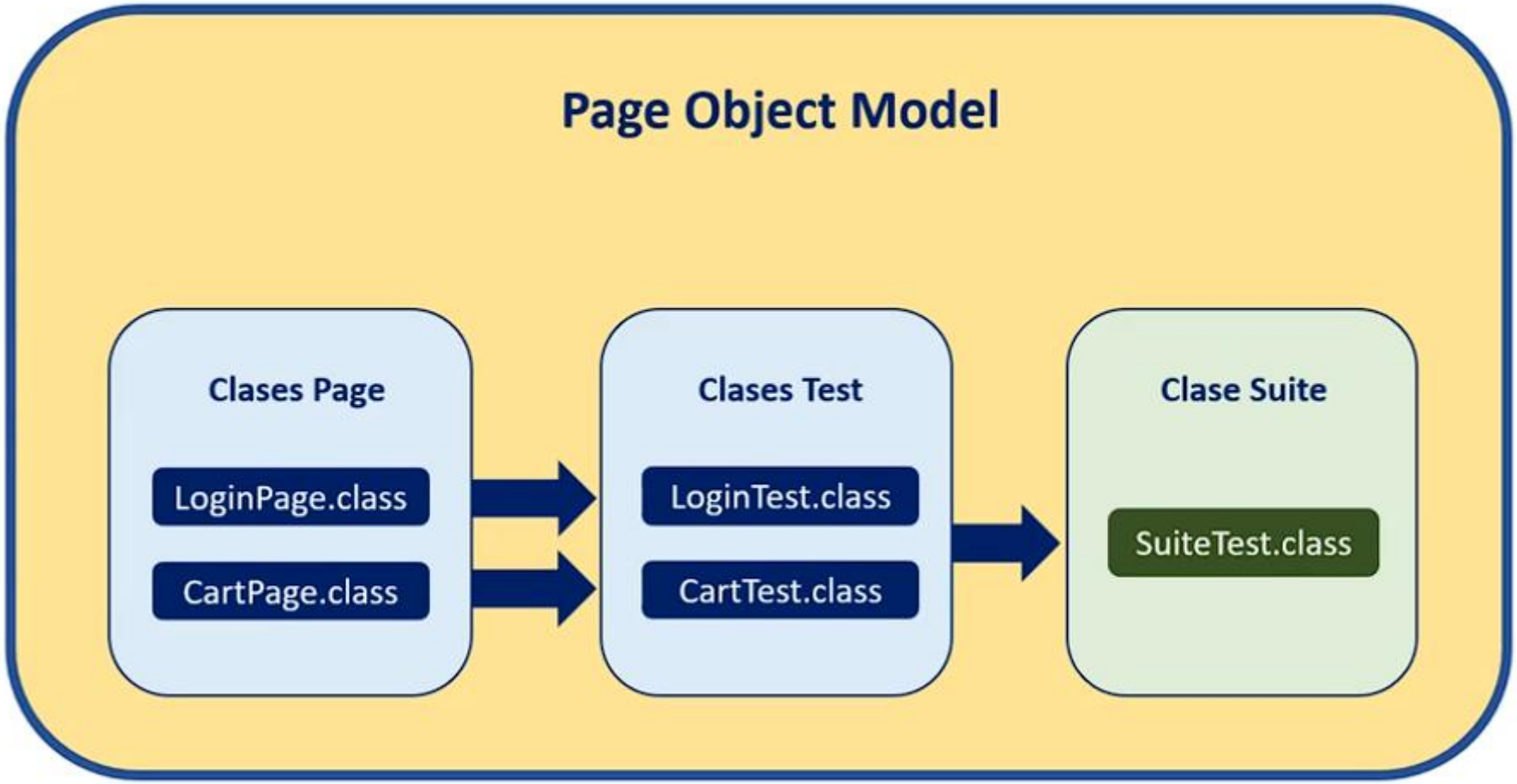
- **By Class Name:** driver.findElement(By.className)
- **By Id:** driver.findElement(By.id)
- **By Xpath:** driver.findElement(By.xpath)
- **By Css Selector:** driver.findElement(By.cssSelector)

## Web Elements




idat

# Page Object Model - POM




idat

# Cucumber y Gherkin

## BDD: Desarrollo orientado al comportamiento

**Lenguaje Gherkin**

```
Feature:   CP01 - Validar inicio de sesión
Scenario:  CP0101 - Probar con credenciales validas
Given      el usuario navega al sitio web
When       ingresa credenciales validas
Then       debería mostrarse el modulo principal
```

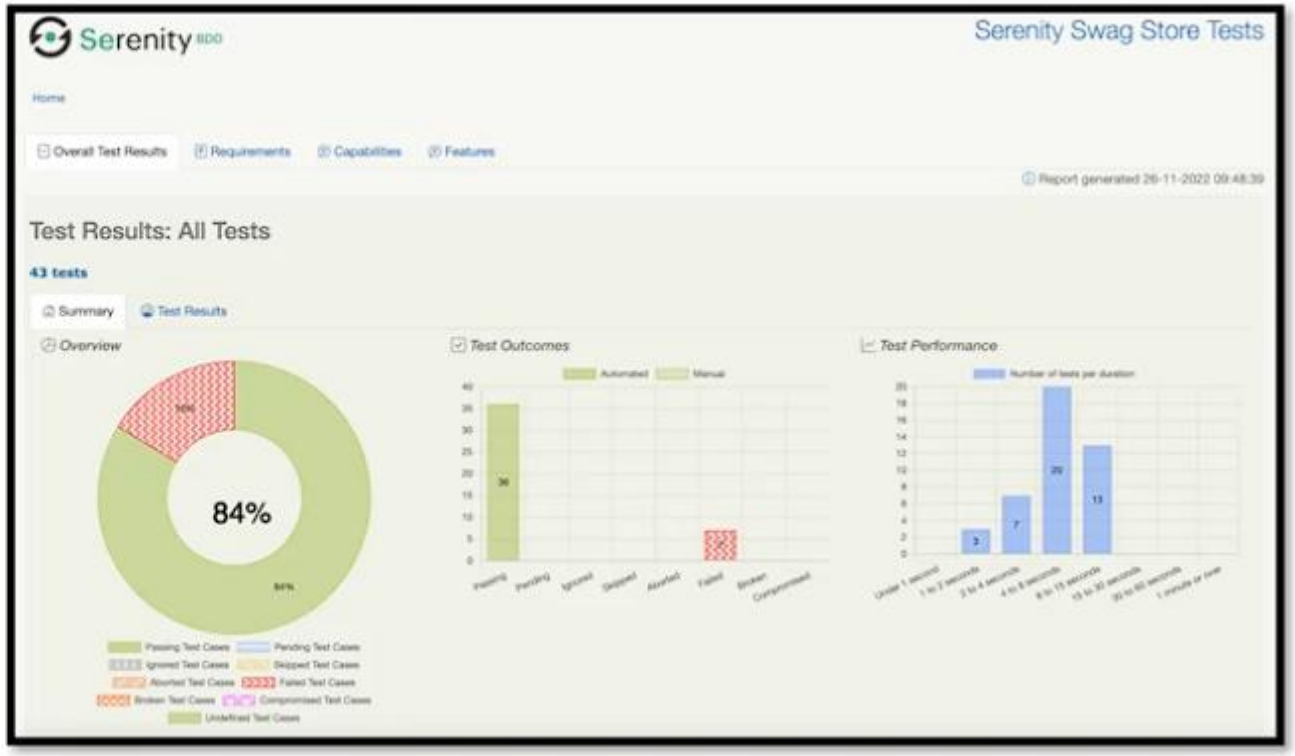
**Cucumber (Steps Definitions)**

```
@When("ingresa credenciales validas")
public void userLogin(){
  login.typeUser("user_demo");
  login.typePass("pass_demo");
  login.clickLogin();
}
```

idat

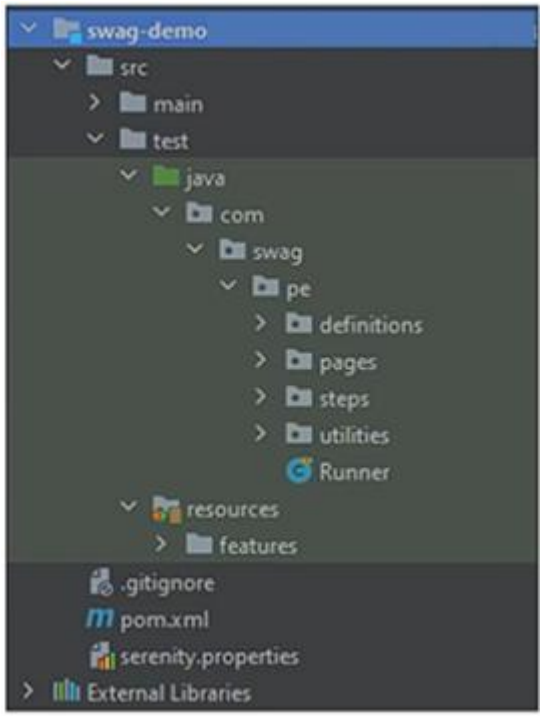
# Serenity BDD

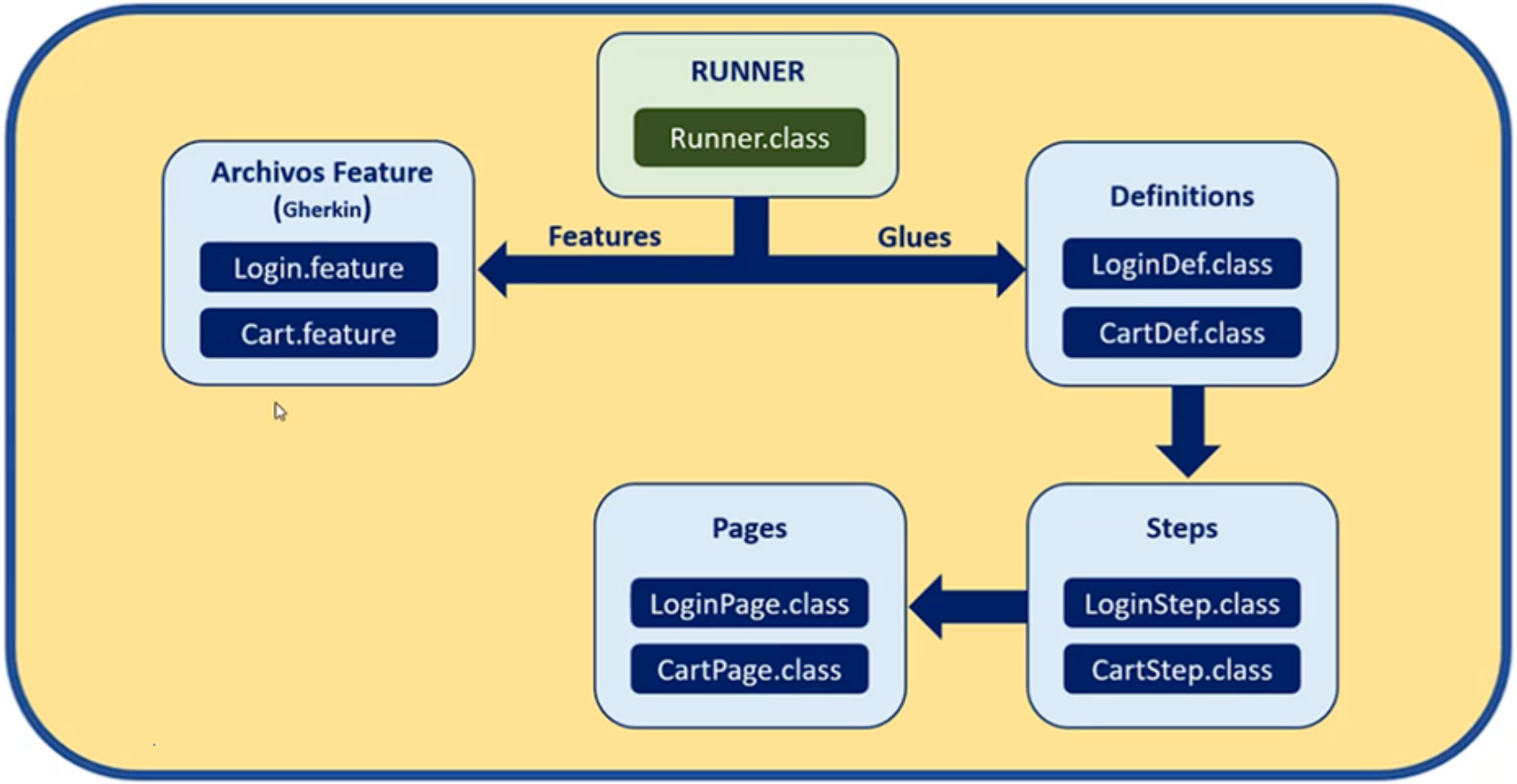
- Ayuda a escribir pruebas de aceptación y regresión automatizadas más limpias y fáciles de mantener.
- Produce informes narrativos ilustrados que documentan y describen lo que hace la aplicación y cómo funciona.




idat

# Estructura de Serenity

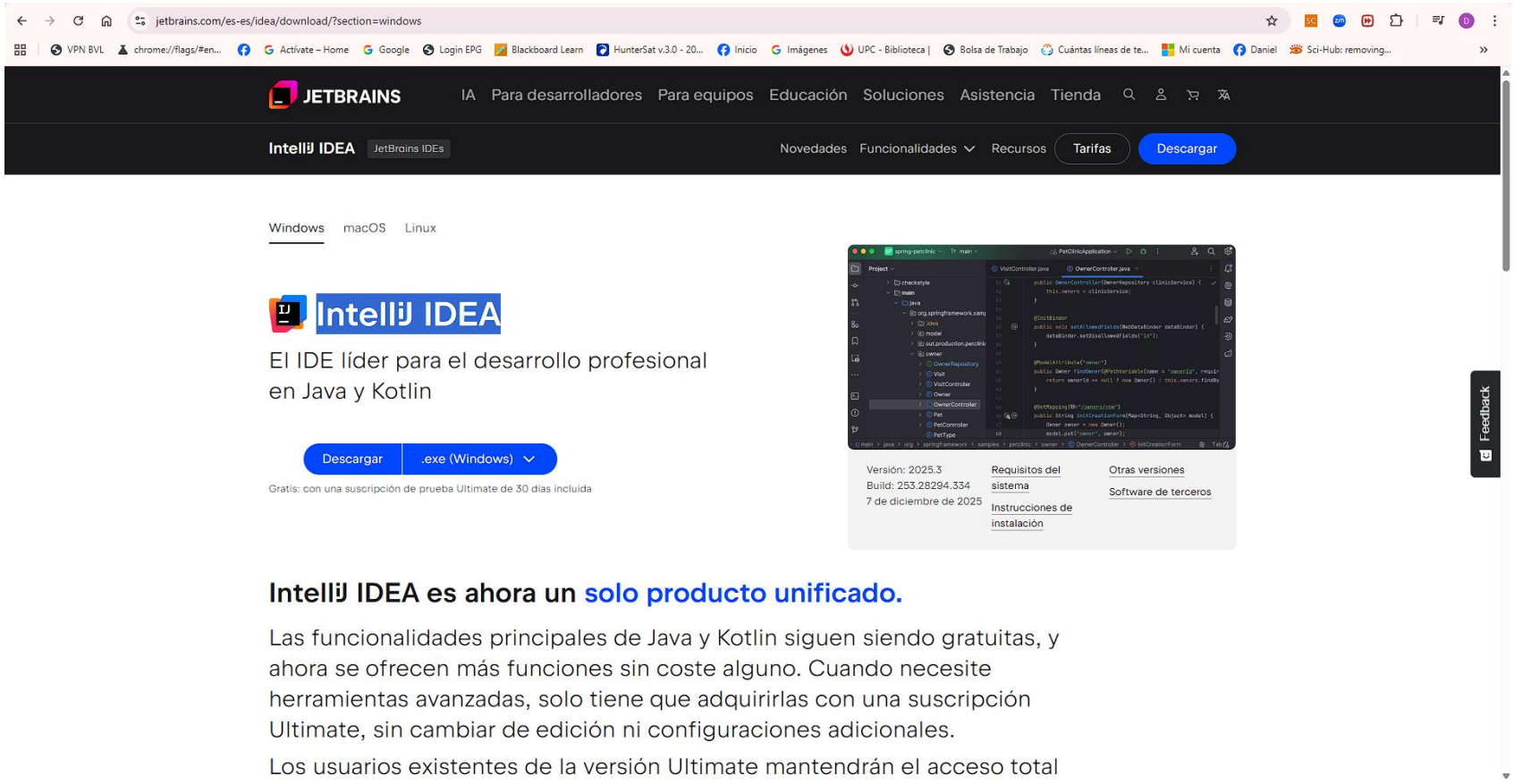



idat

# IntelliJ IDEA

Ingresamos al link https://www.jetbrains.com/es-es/idea/download/?section=windows




idat

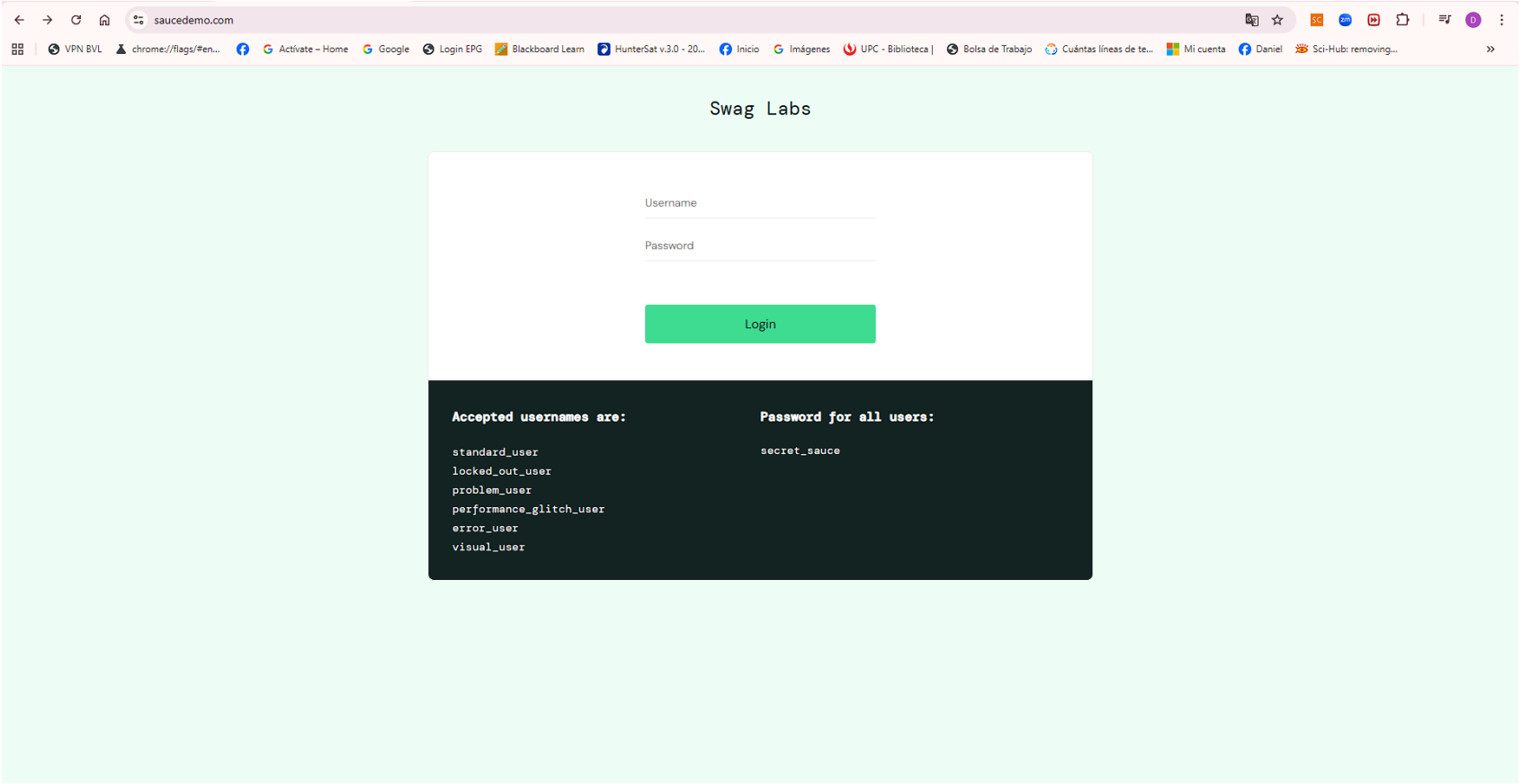
# Laboratorio: Cucumber




idat

# Nuestra página de pruebas

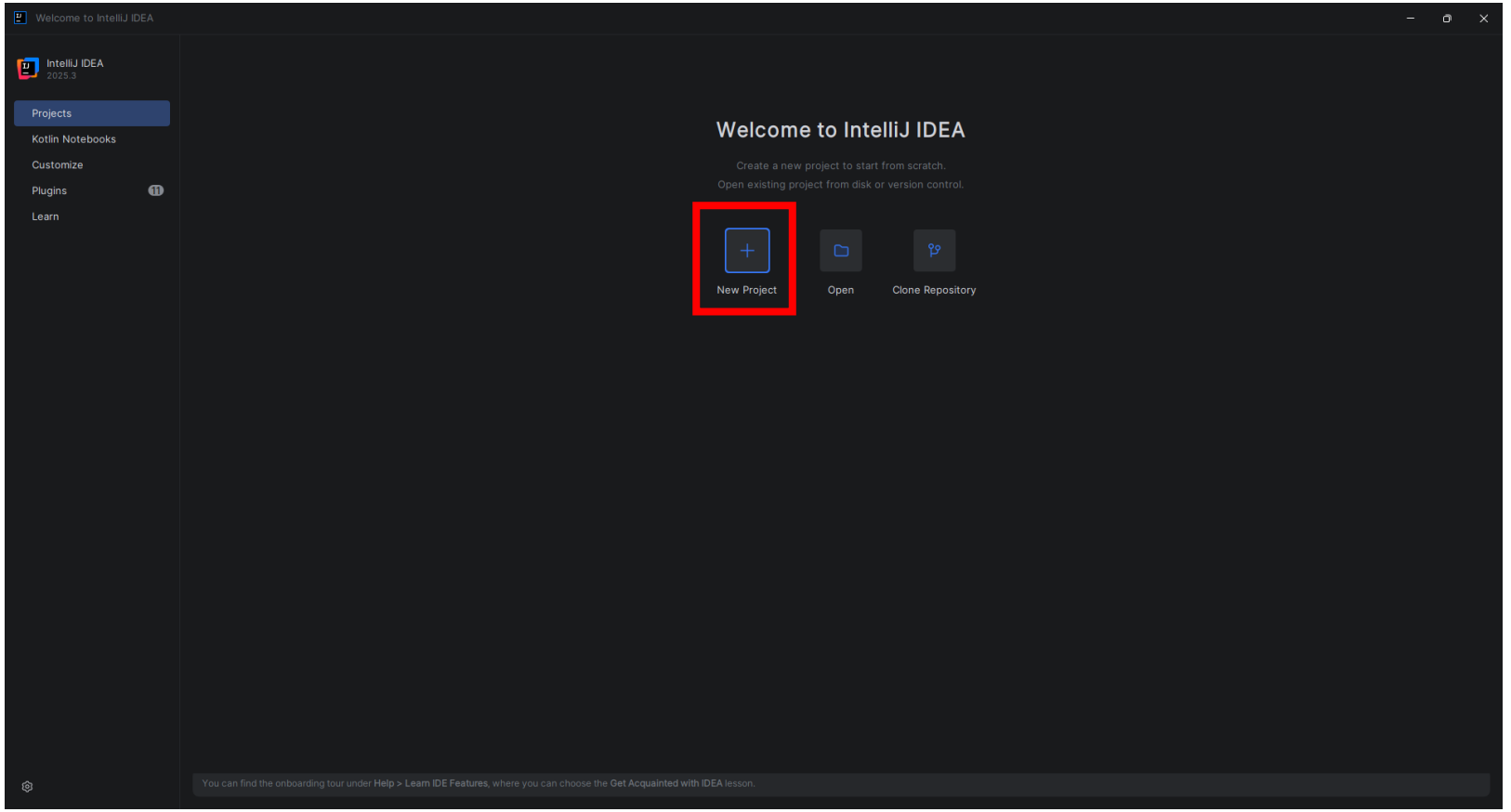
Ingresamos al link https://www.saucedemo.com/




idat

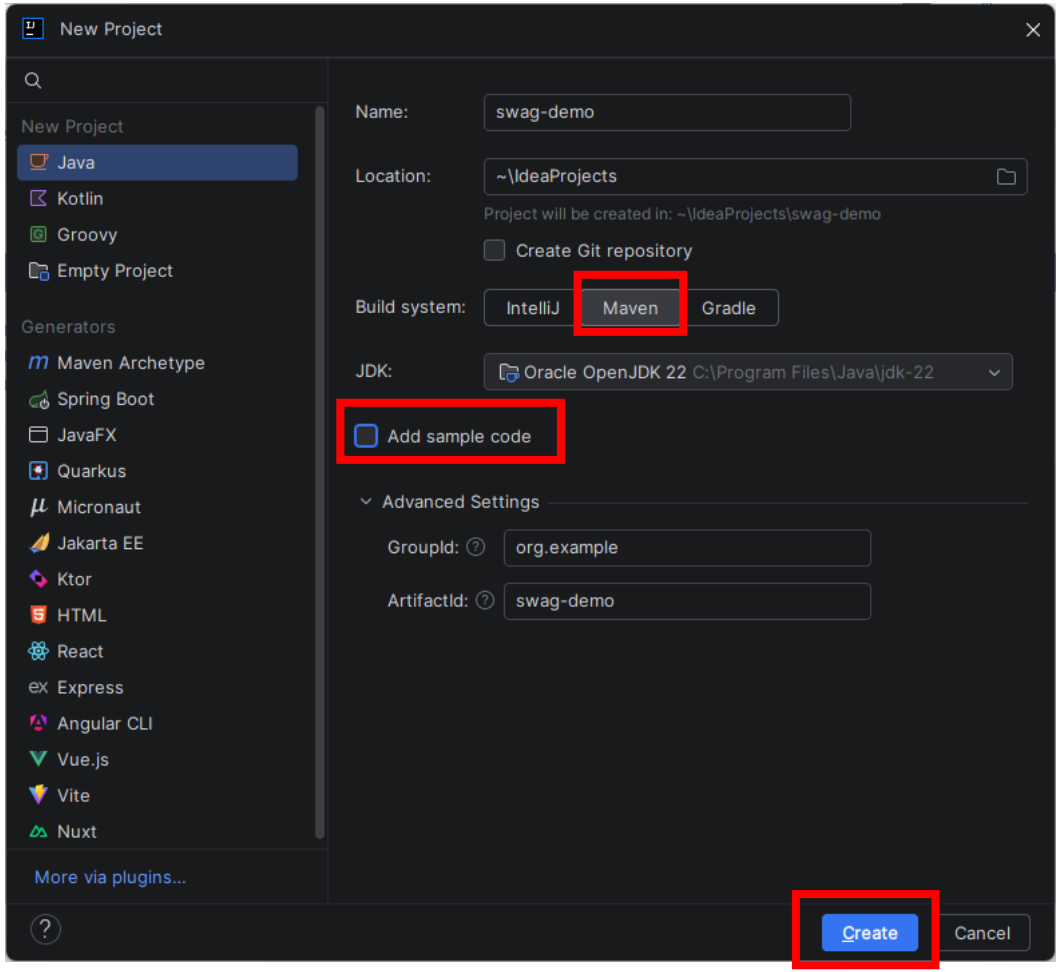
# Ejecutar IntelliJ IDEA

Creamos el Proyecto “swag-demo”




idat

# Ejecutar IntelliJ IDEA




idat

# Modificar el pom.xml

```xml
    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <serenity-version>3.9.0</serenity-version>
        <junit-version>4.13.2</junit-version>
        <maven-plugin-version>3.1.2</maven-plugin-version>
        <slf4j-version>2.0.7</slf4j-version>
    </properties>
```

```xml
    <dependencies>
        <dependency>
            <groupId>net.serenity-bdd</groupId>
            <artifactId>serenity-core</artifactId>
            <version>${serenity-version}</version>
        </dependency>
        <dependency>
            <groupId>net.serenity-bdd</groupId>
            <artifactId>serenity-cucumber</artifactId>
            <version>${serenity-version}</version>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>net.serenity-bdd</groupId>
            <artifactId>serenity-junit</artifactId>
            <version>${serenity-version}</version>
        </dependency>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>${junit-version}</version>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.slf4j</groupId>
            <artifactId>slf4j-simple</artifactId>
            <version>${slf4j-version}</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
```

https://mvnrepository.com/

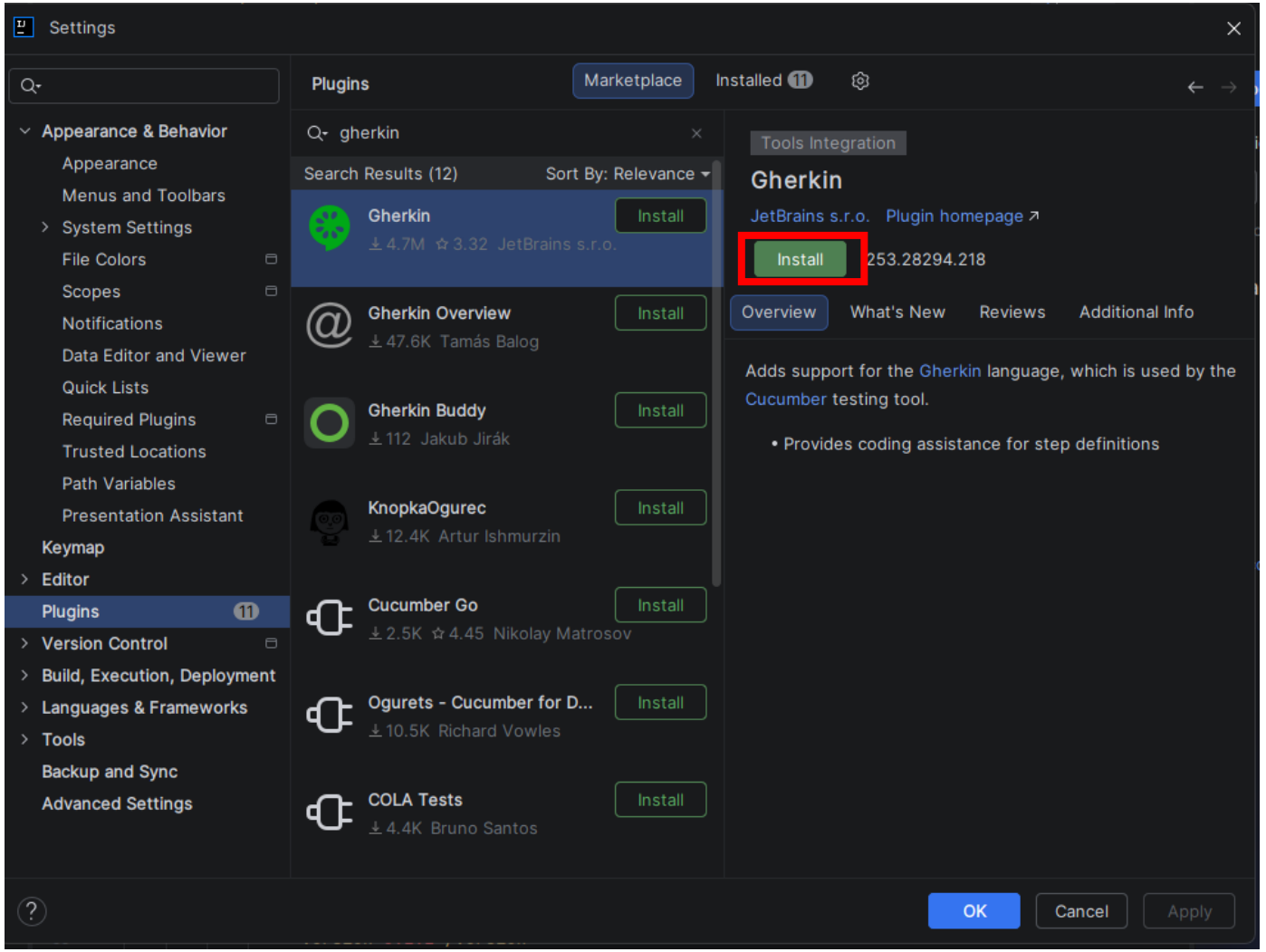
idat

# Modificar el pom.xml

```xml
<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>${maven-plugin-version}</version>
            <configuration>
                <includes>
                    <include>**/Runner*.java</include>
                </includes>
            </configuration>
        </plugin>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-failsafe-plugin</artifactId>
            <version>${maven-plugin-version}</version>
            <configuration>
                <includes>
                    <include>**/test/**/*.java</include>
                </includes>
                <systemPropertyVariables>
                    <surefire.security.manager>
                        java.lan.SecurityManager
                    </surefire.security.manager>
                </systemPropertyVariables>
            </configuration>
            <executions>
                <execution>
                    <goals>
                        <goal>integration-test</goal>
                        <goal>verify</goal>
                    </goals>
                </execution>
            </executions>
        </plugin>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>${serenity-version}</version>
            <configuration>
                <source>${maven.compiler.source}</source>
                <source>${maven.compiler.target}</source>
            </configuration>
        </plugin>
        <plugin>
            <groupId>net.serenity-bdd.maven.plugins</groupId>
            <artifactId>serenity-maven-plugin</artifactId>
            <version>${serenity-version}</version>
            <executions>
                <execution>
                    <id>serenity-reports</id>
                    <phase>post-integration-test</phase>
                    <goals>
                        <goal>aggregate</goal>
                    </goals>
                </execution>
            </executions>
        </plugin>
    </plugins>
</build>
```

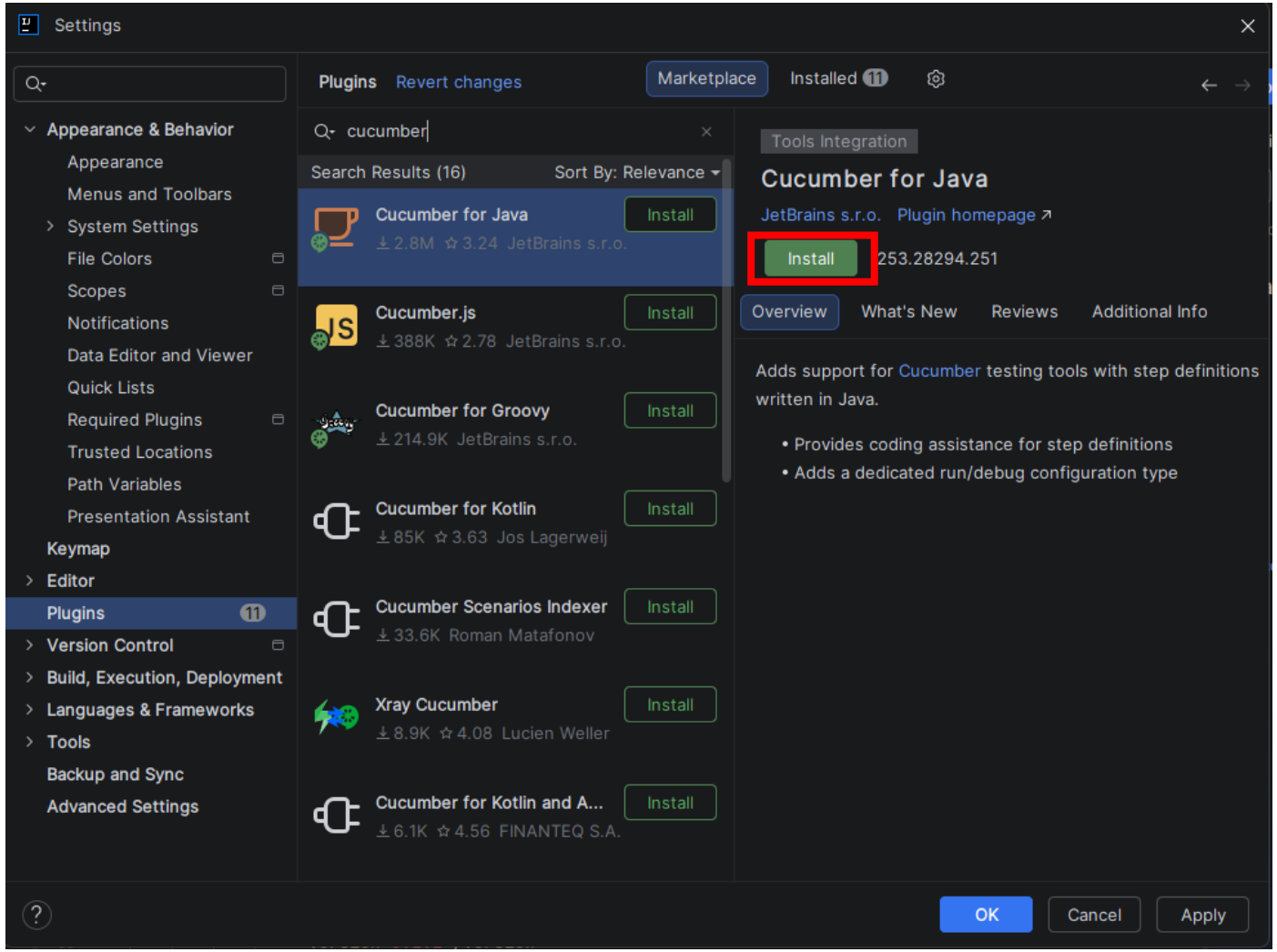
idat

# Instalamos el plugin de Gherkin




idat

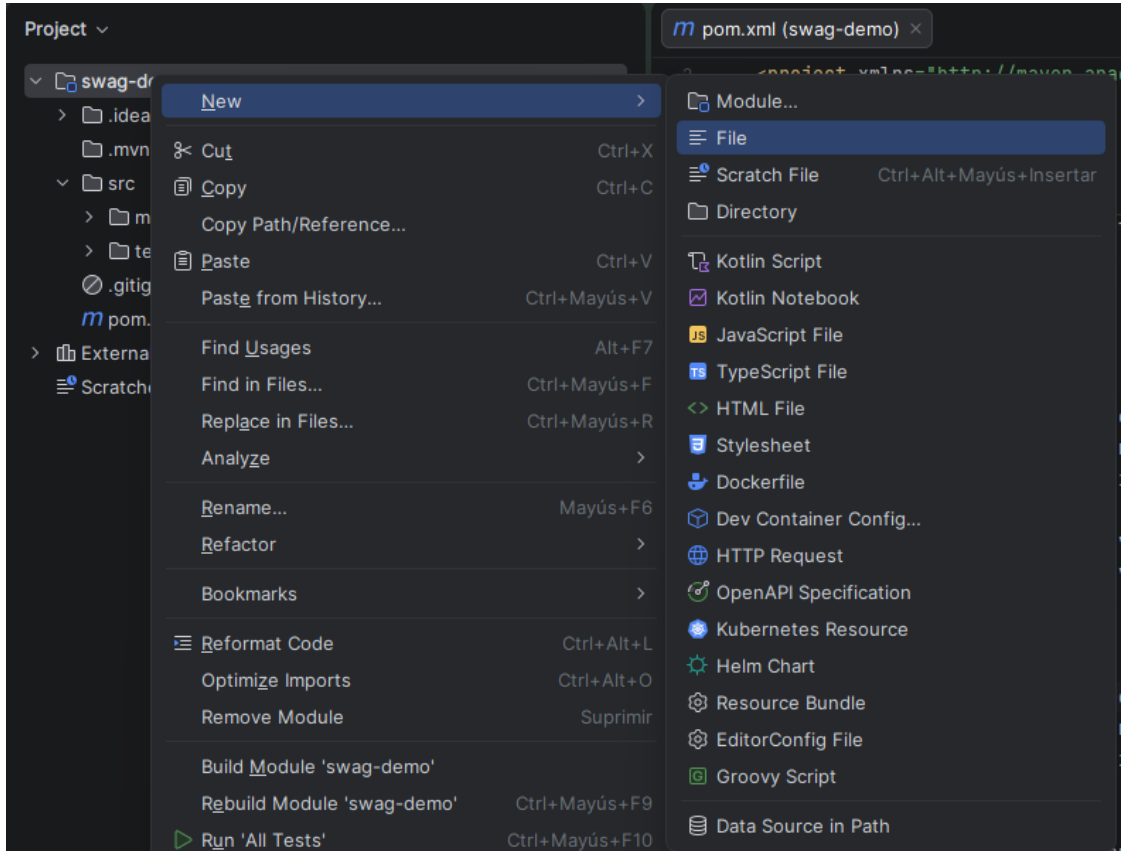
# Instalamos el plugin de Cucumber

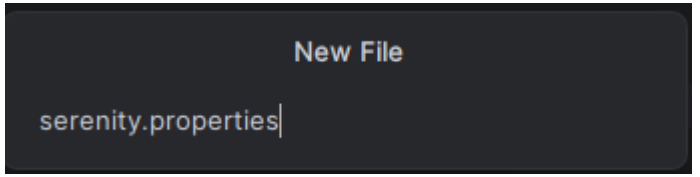



idat

# Creamos el archivo: serenity.properties

Al nivel del Proyecto, creamos el archivo: serenity.properties



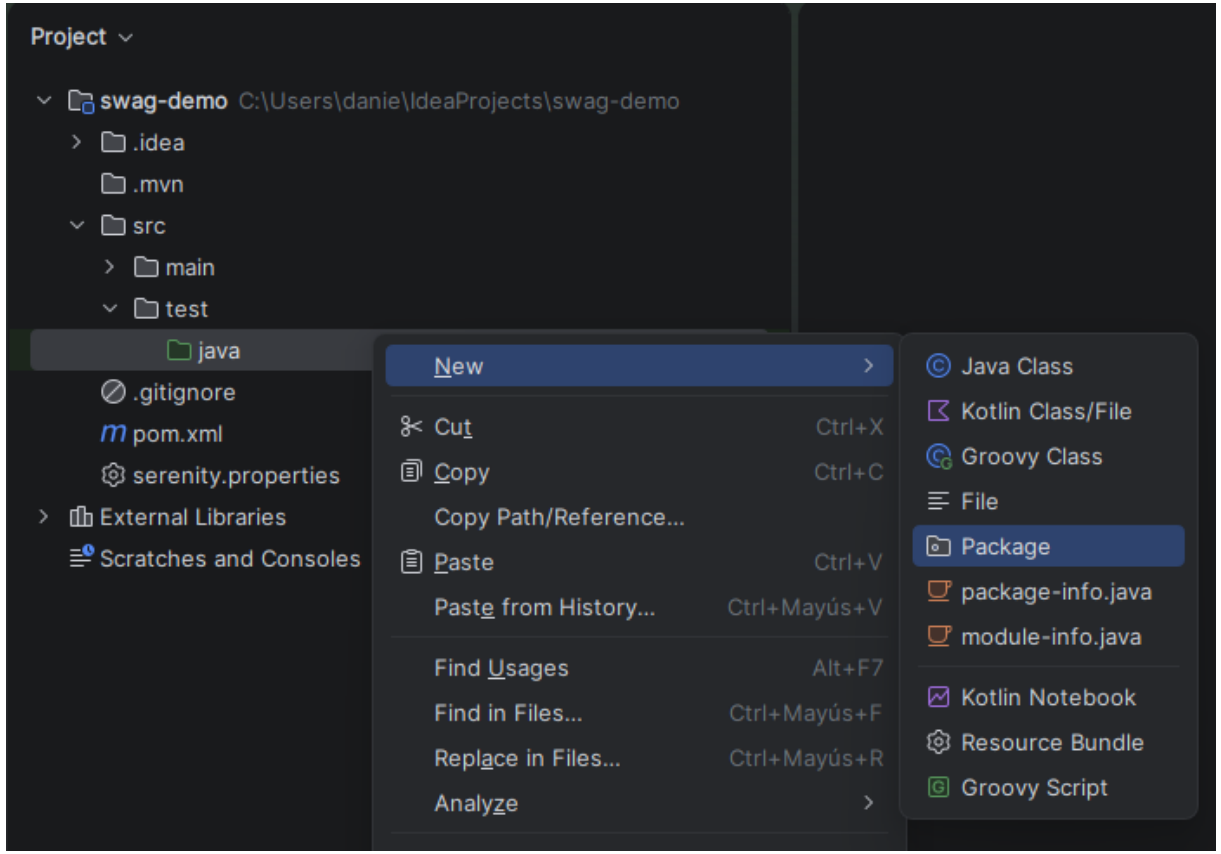


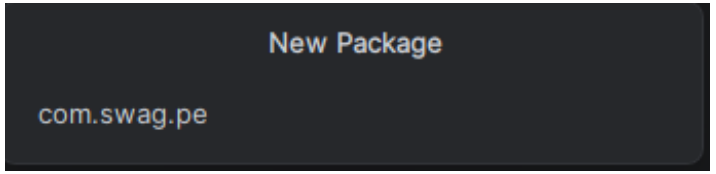


idat

# Creamos la estructura del Proyecto

Dentro de src.test.java, creamos el paquete: com.swag.pe

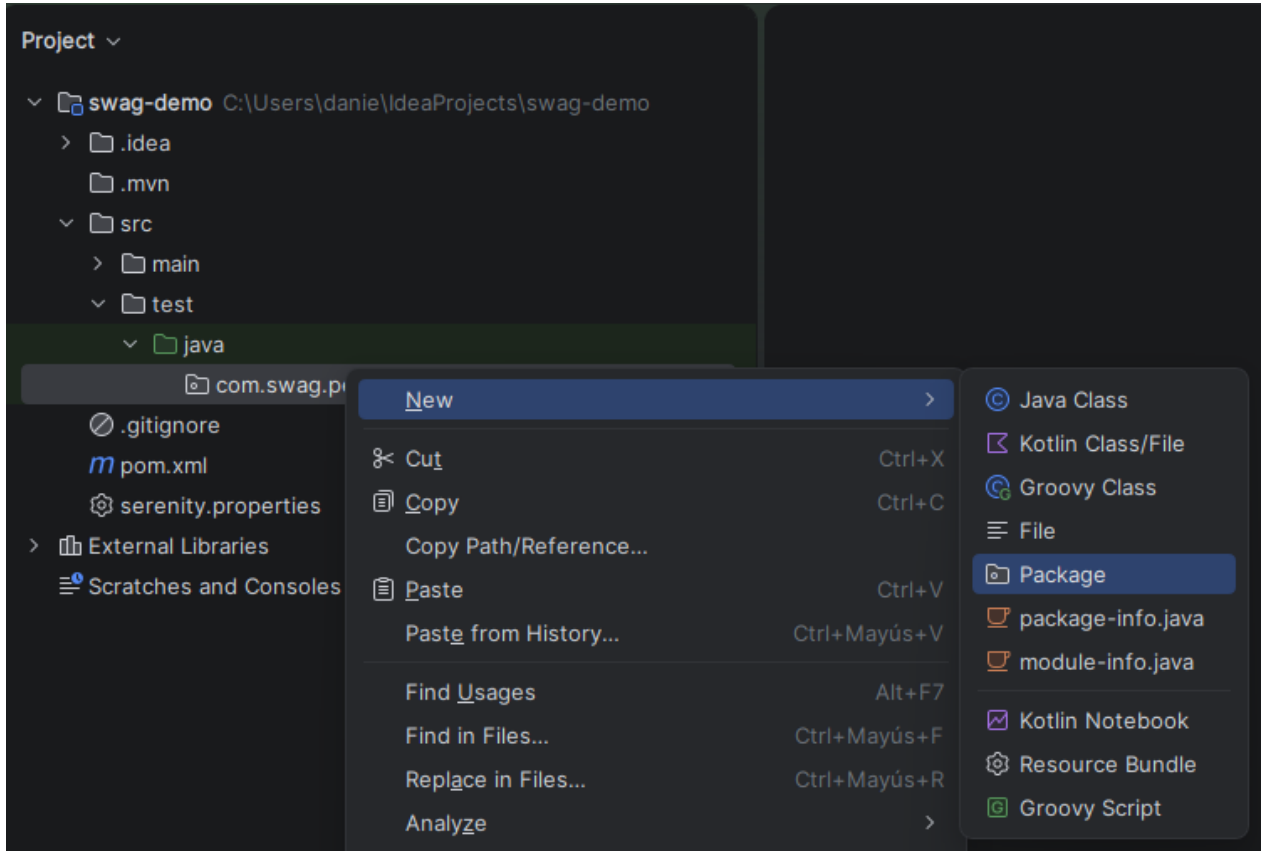


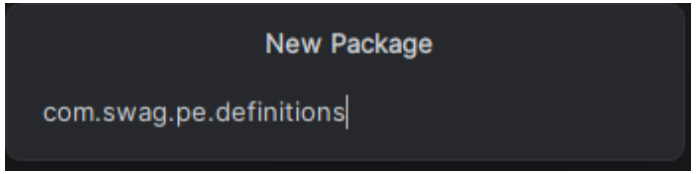




idat

# Creamos la estructura del Proyecto

Dentro de com.swag.pe, creamos el paquete: definitions

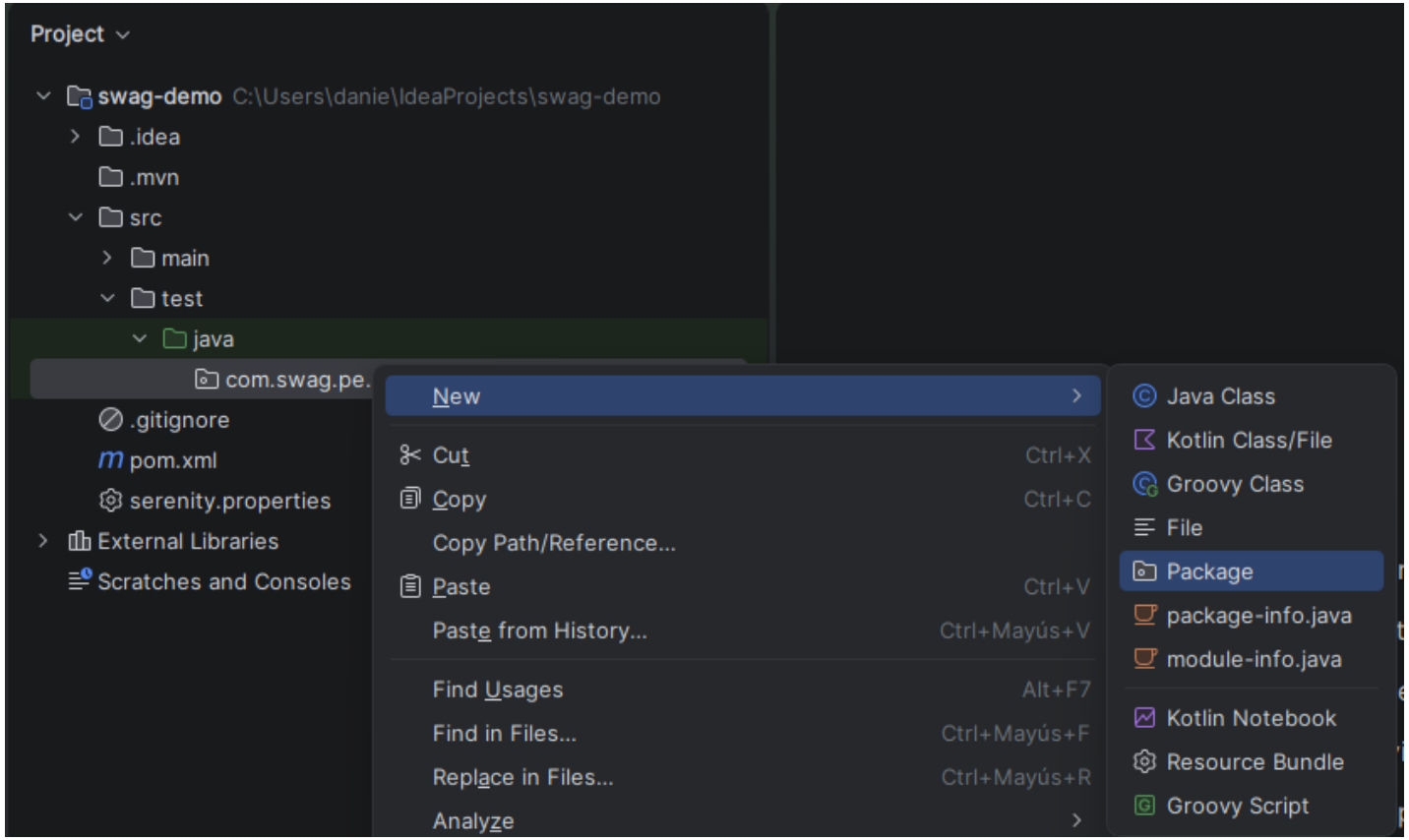


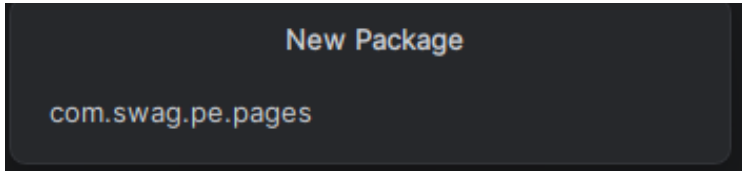




idat

# Creamos la estructura del Proyecto

Dentro de com.swag.pe, creamos el paquete: pages

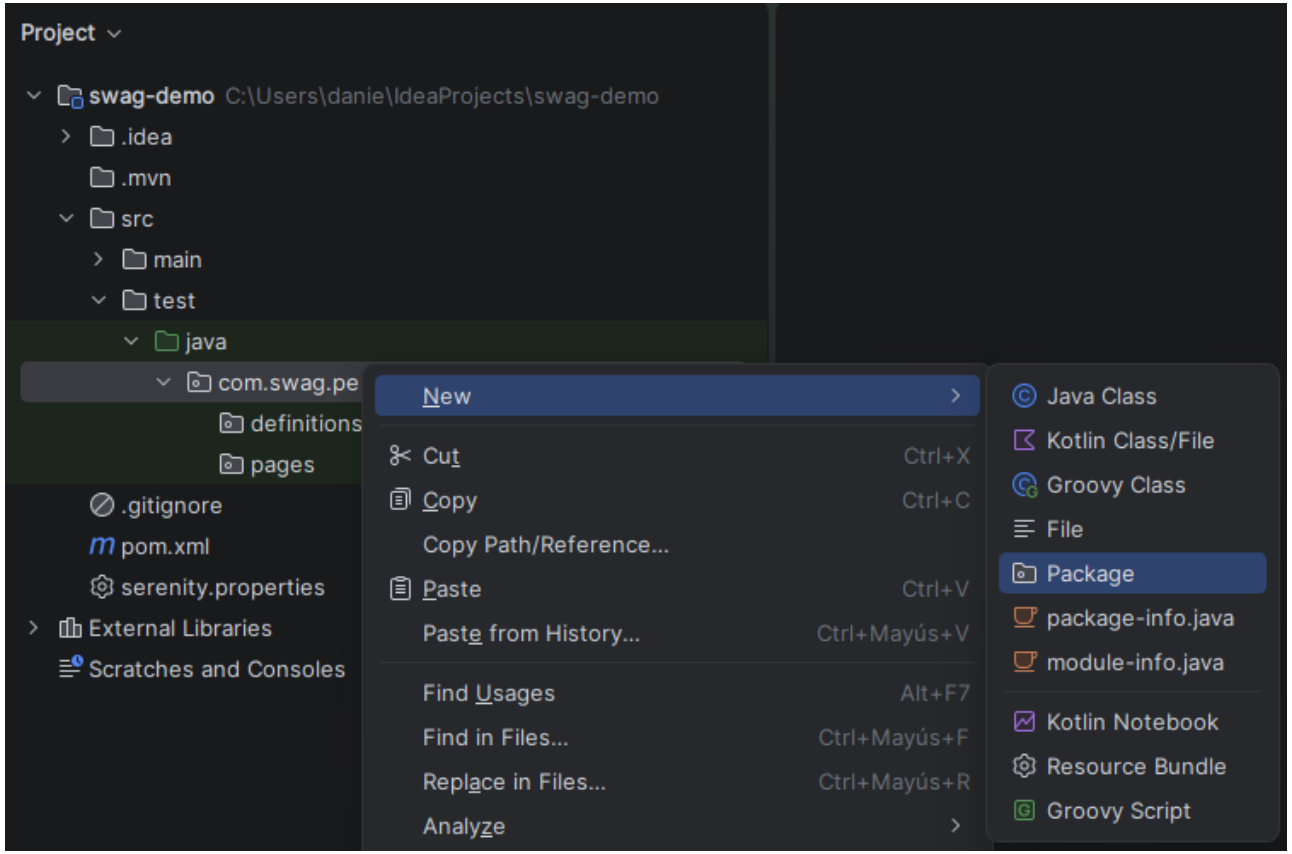


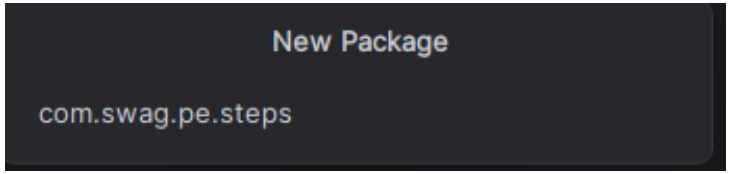




idat

# Creamos la estructura del Proyecto

Dentro de com.swag.pe, creamos el paquete: steps

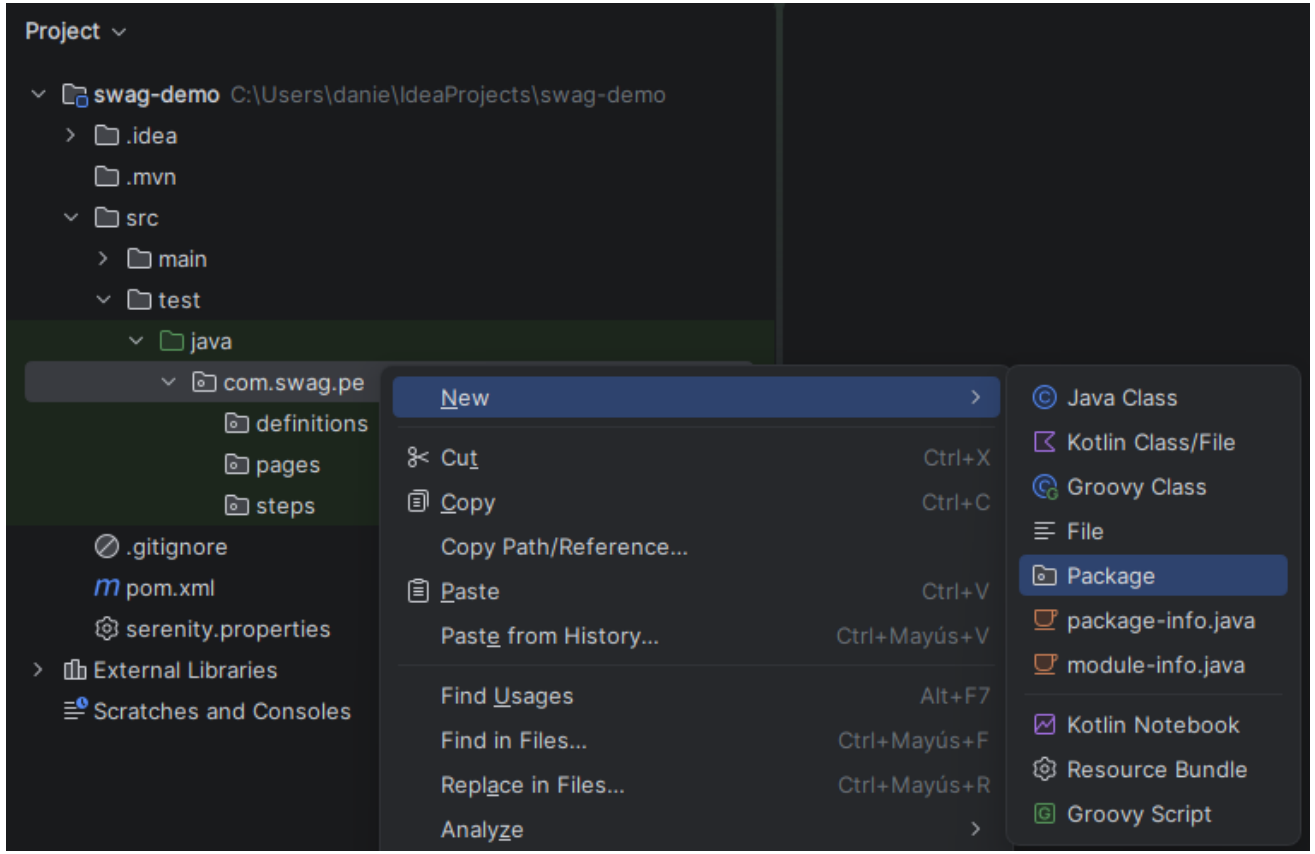


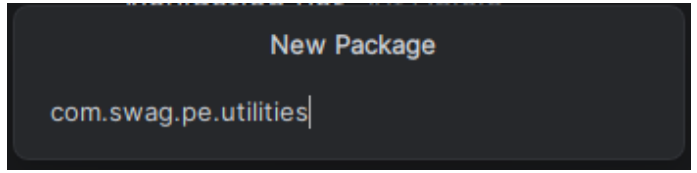




idat

# Creamos la estructura del Proyecto

Dentro de com.swag.pe, creamos el paquete: utilities

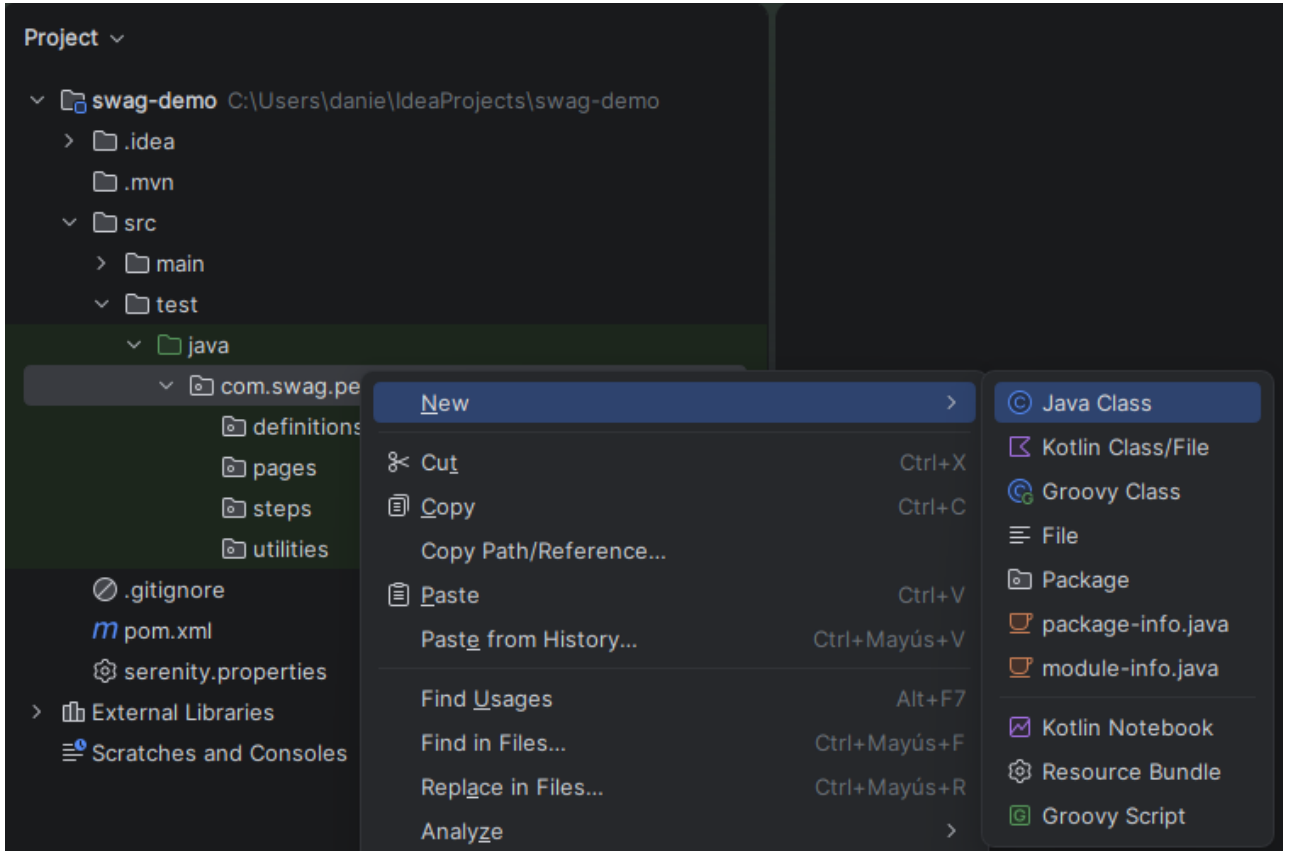


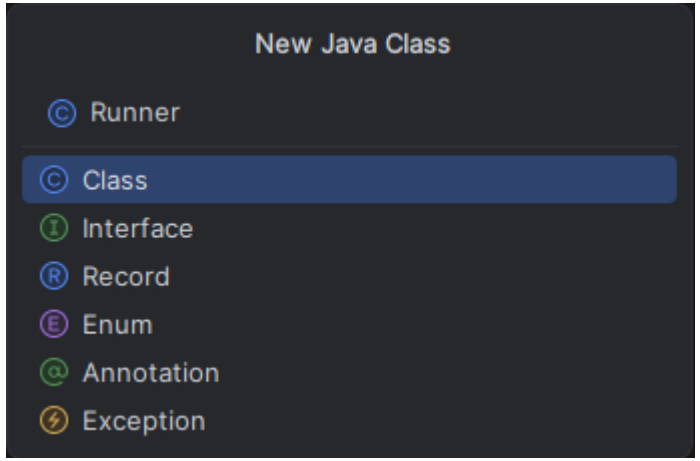




idat

# Creamos la Clase: Runner

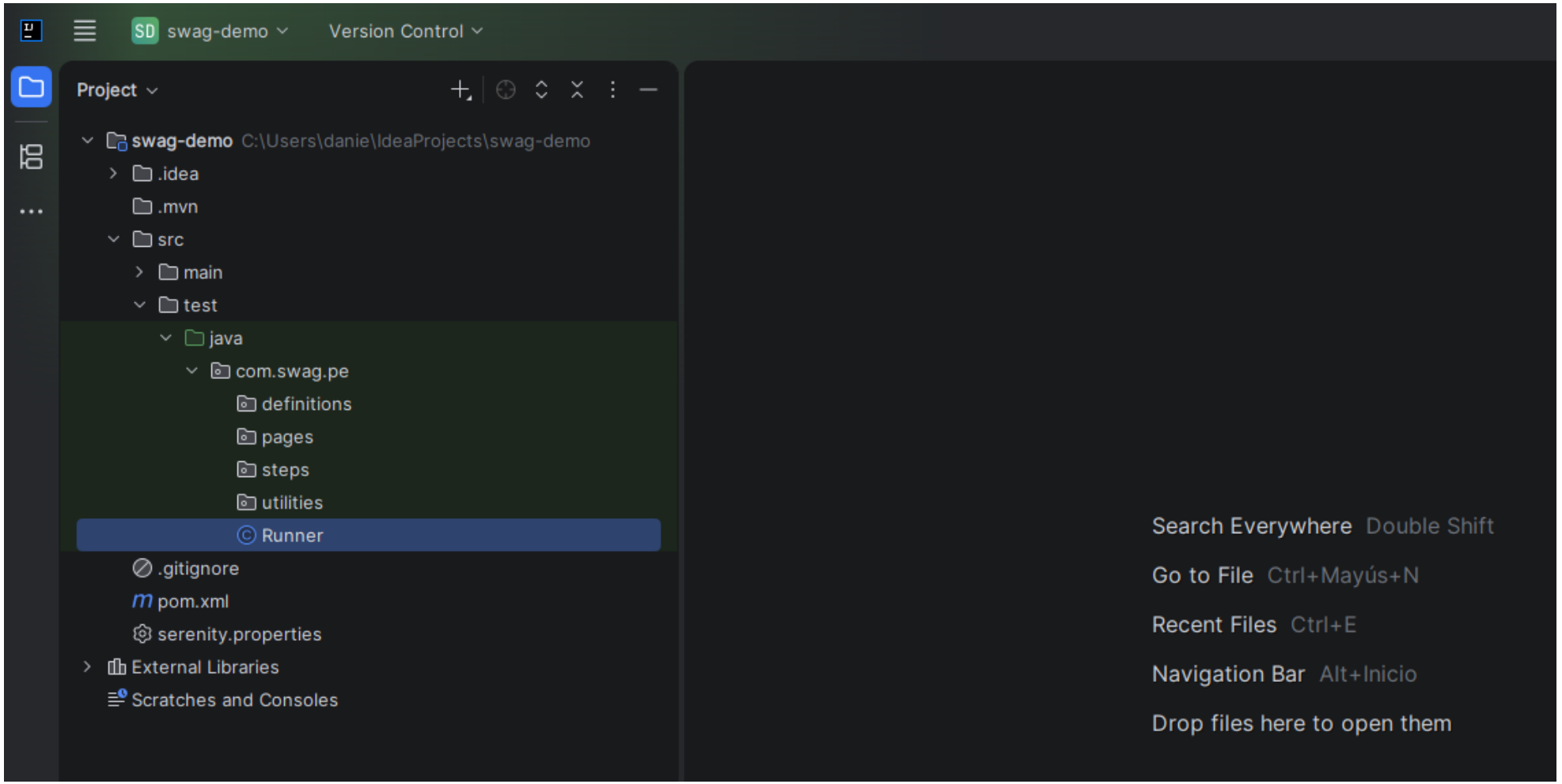
Dentro de com.swag.pe, creamos la Clase: Runner




idat

# Tenemos lista la estructura




idat

# Iniciamos con las pruebas

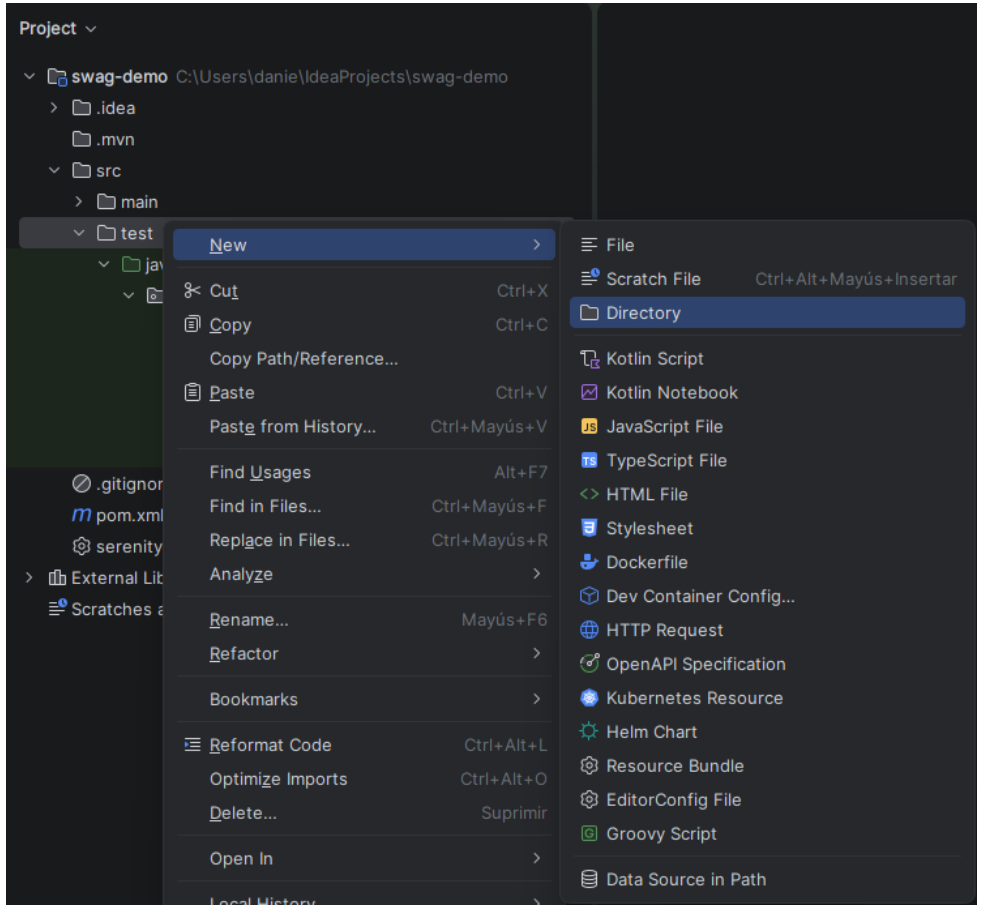
Debemos configurar nuestro escenario de pruebas

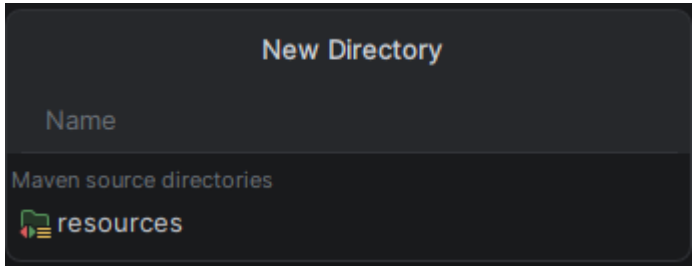



idat

# Creamos el Directorio: Resources

Dentro de test, creamos el Directorio: resources

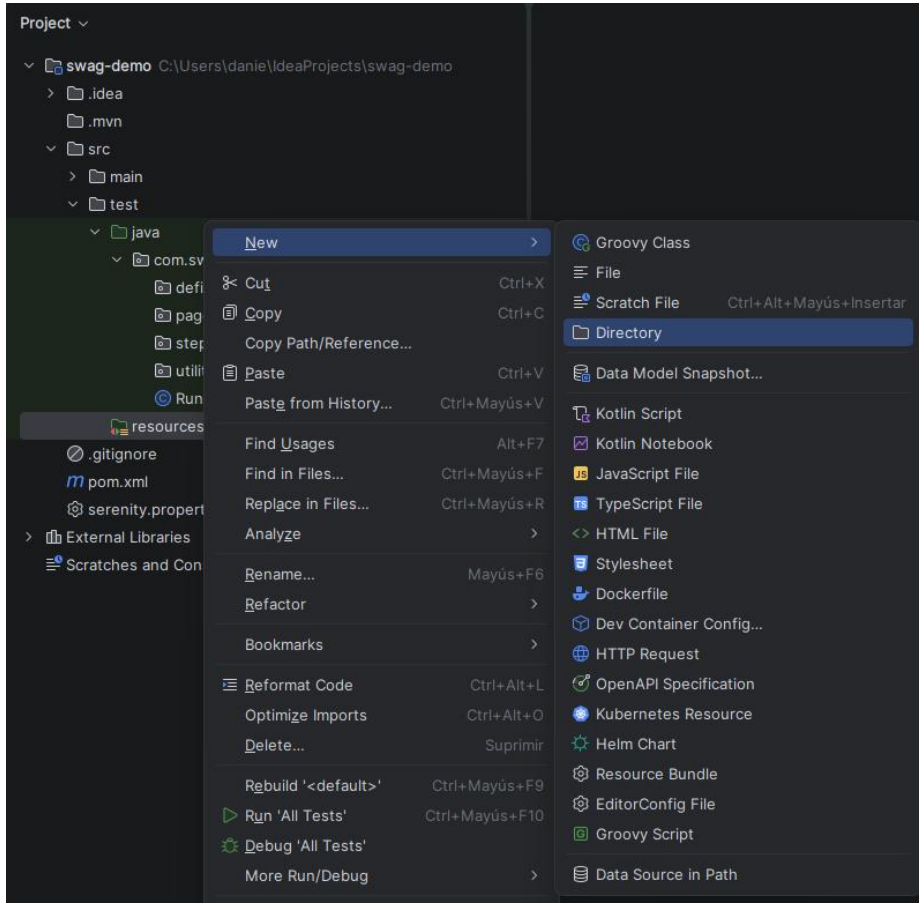


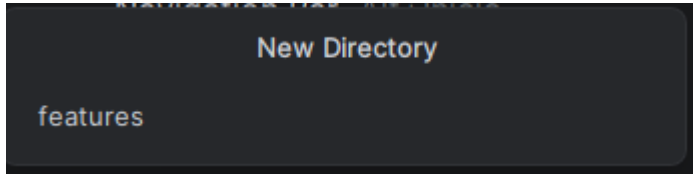




idat

# Creamos el Directorio: Features

Dentro del directorio resources, creamos el Directorio: features

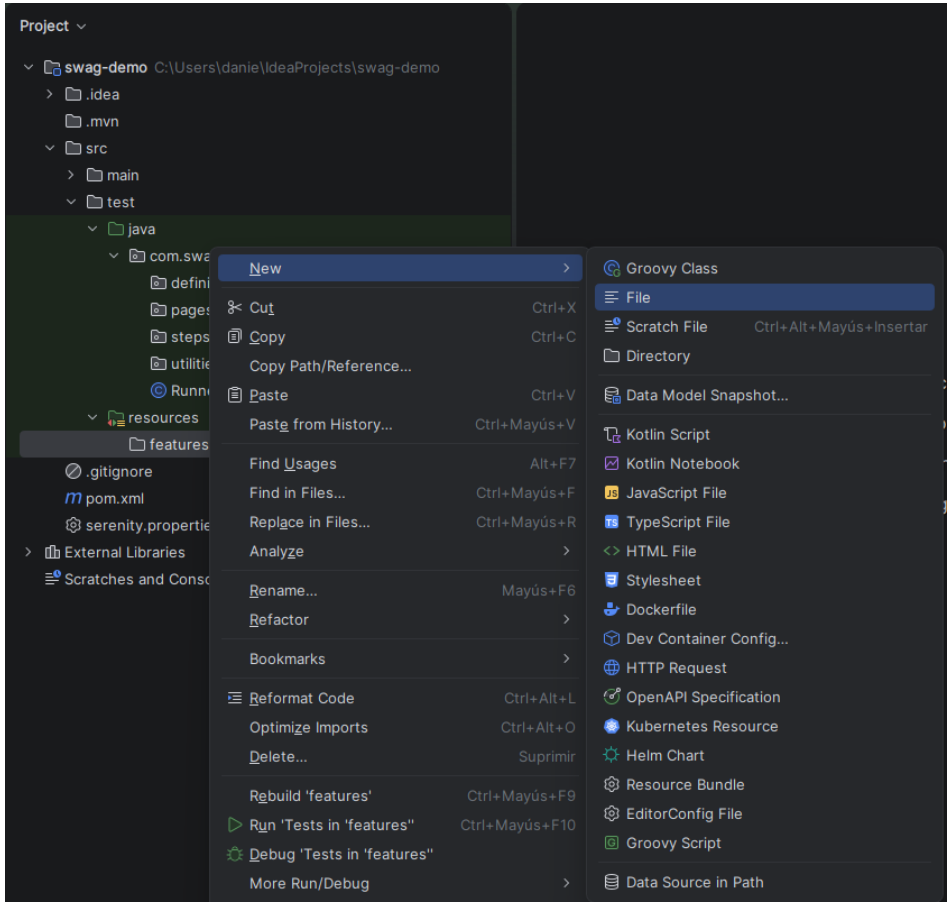


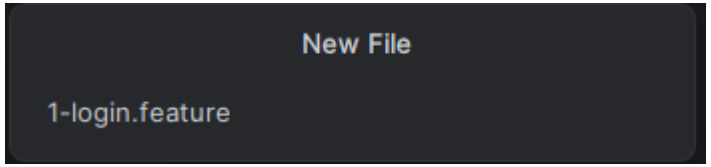




idat

# Creamos el Feature: login

Dentro del directorio features, creamos el Archivo: 1-login.feature




idat

# Llenamos el Feature: login

1-login.feature

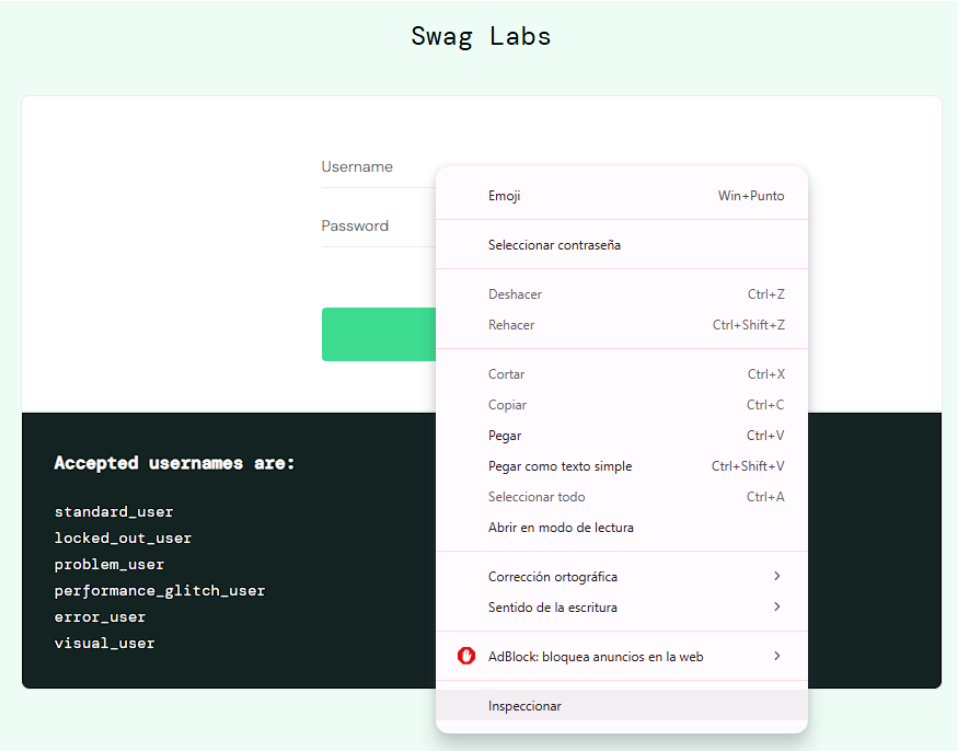
```
1  Feature: CP01 - Validar inicio de sesion
2
3    Background: Validar el inicio de sesion con credenciales validas e invalidas
4
5      Given el usuario navega al sitio web
6
7      Scenario: 1 - Validar con credenciales correctas
8        When ingresa credenciales validas
9        Then la aplicacion deberia mostrar el modulo principal de productos
10
11     Scenario: 2 - Validar con credenciales incorrectas
12       When ingresa credenciales invalidas
13       Then la aplicacion deberia mostrar un mensaje de error
```

idat

# Debemos ubicar los elementos

Debemos ubicar los elementos de la página web, para poder referenciarlos

- Ingresamos al link https://www.saucedemo.com/
- Hacemos click en “username”, botón derecho y elegimos inspeccionar



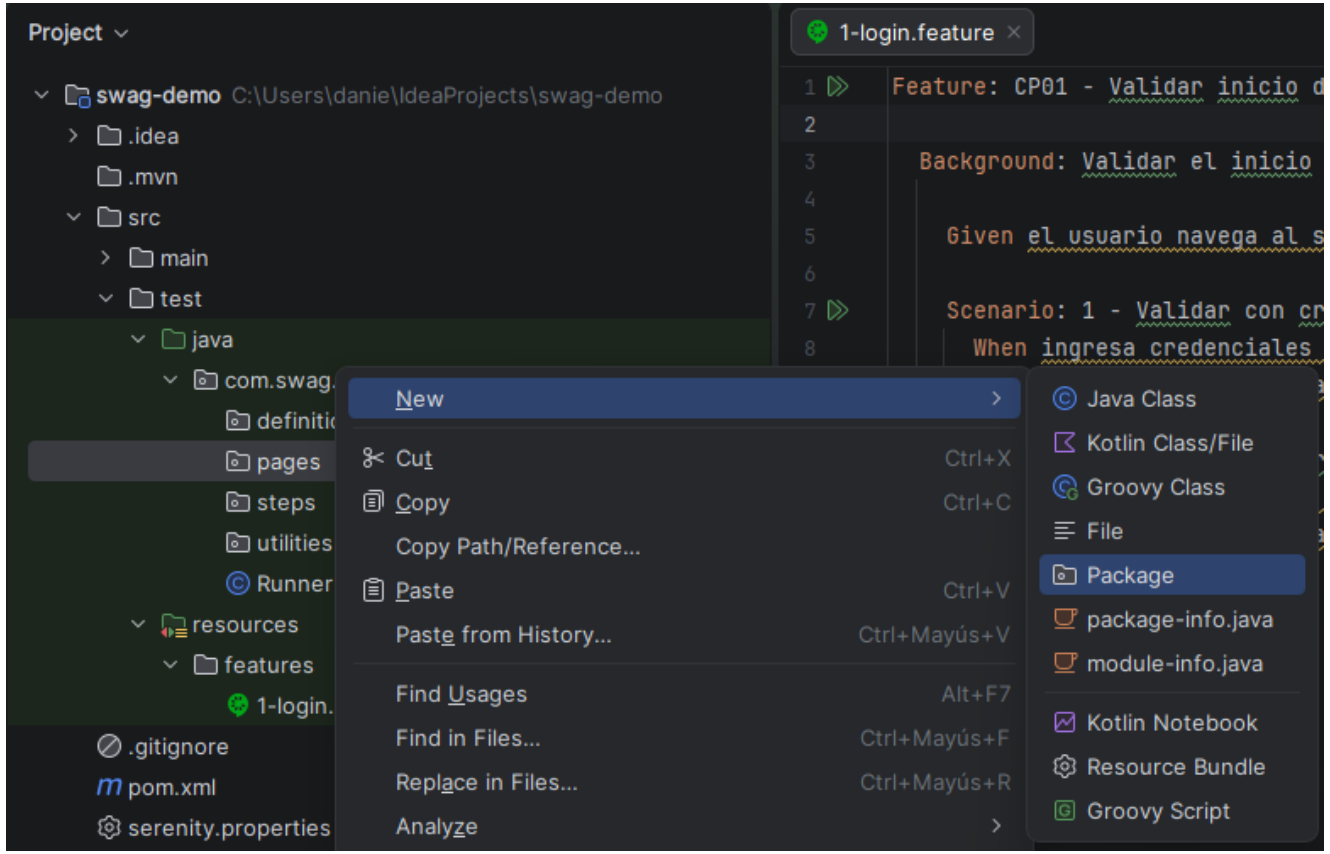

- Buscamos si existe el tag “id”

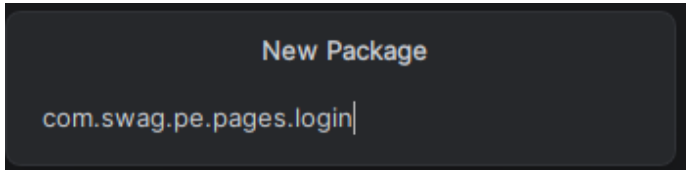
```
<div class="form_group">
  <input class="input_error form_input" placeholder="Username"
  type="text" data-test="username" id="user-name" name="user-na
  me" autocorrect="off" autocapitalize="none" value> == $0
```

idat

# Creamos el paquete Login

Dentro de com.swag.pe.pages, creamos el paquete: login

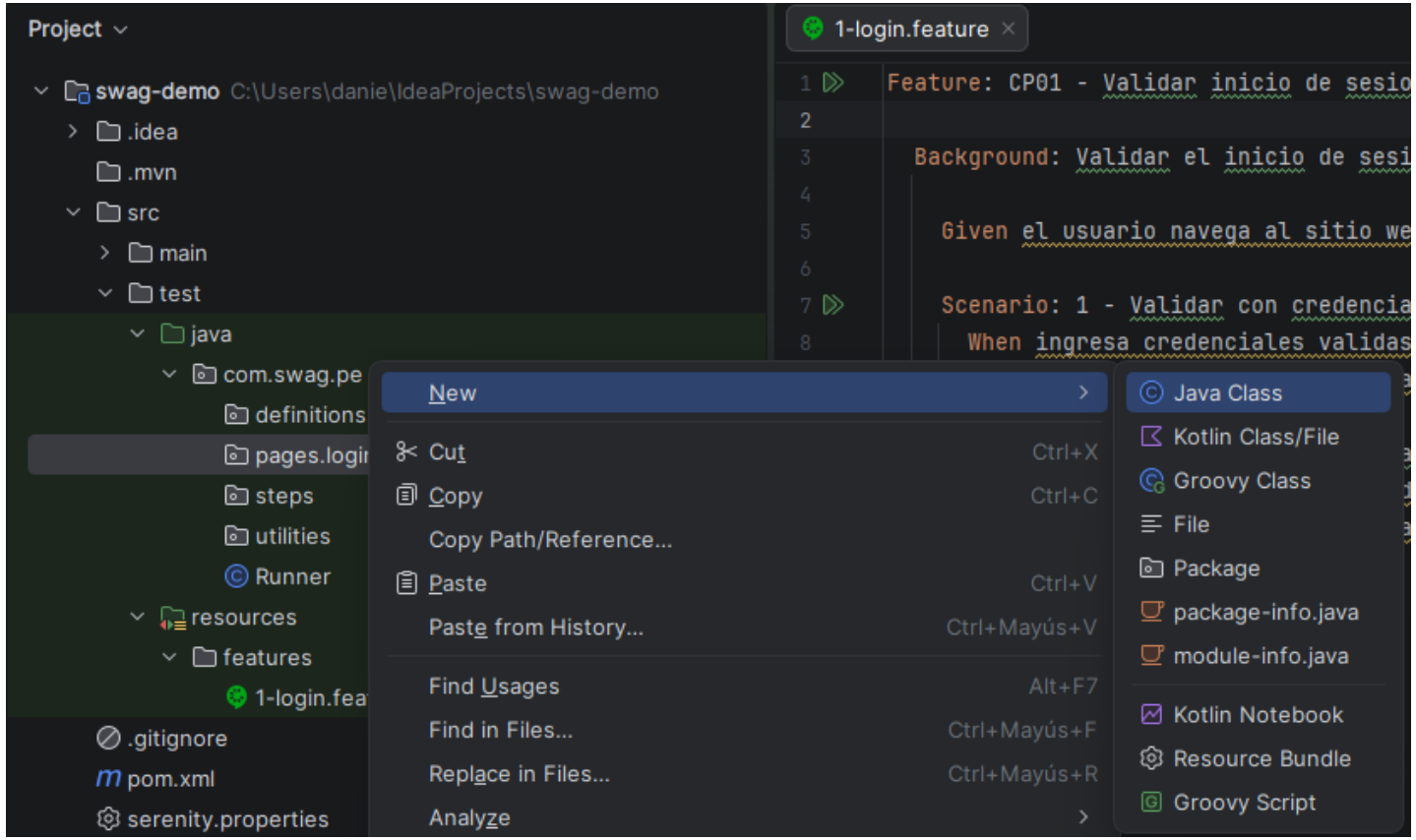


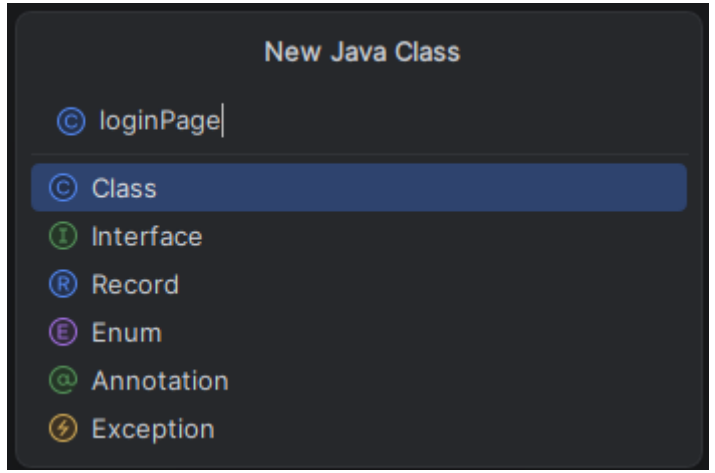




idat

# Creamos la clase LoginPage

Dentro de com.swag.pe.pages.login, creamos la clase: loginPage




idat

# Codificamos clase LoginPage

La extendemos de la clase PageObject

1-login.feature | loginPage.java

```java
package com.swag.pe.pages.login;

import net.serenitybdd.core.pages.PageObject;
import net.serenitybdd.core.pages.WebElementFacade;
import org.openqa.selenium.support.FindBy;

public class loginPage extends PageObject {  no usages

    @FindBy(id = "user-name")  no usages
    protected WebElementFacade txt_userName;

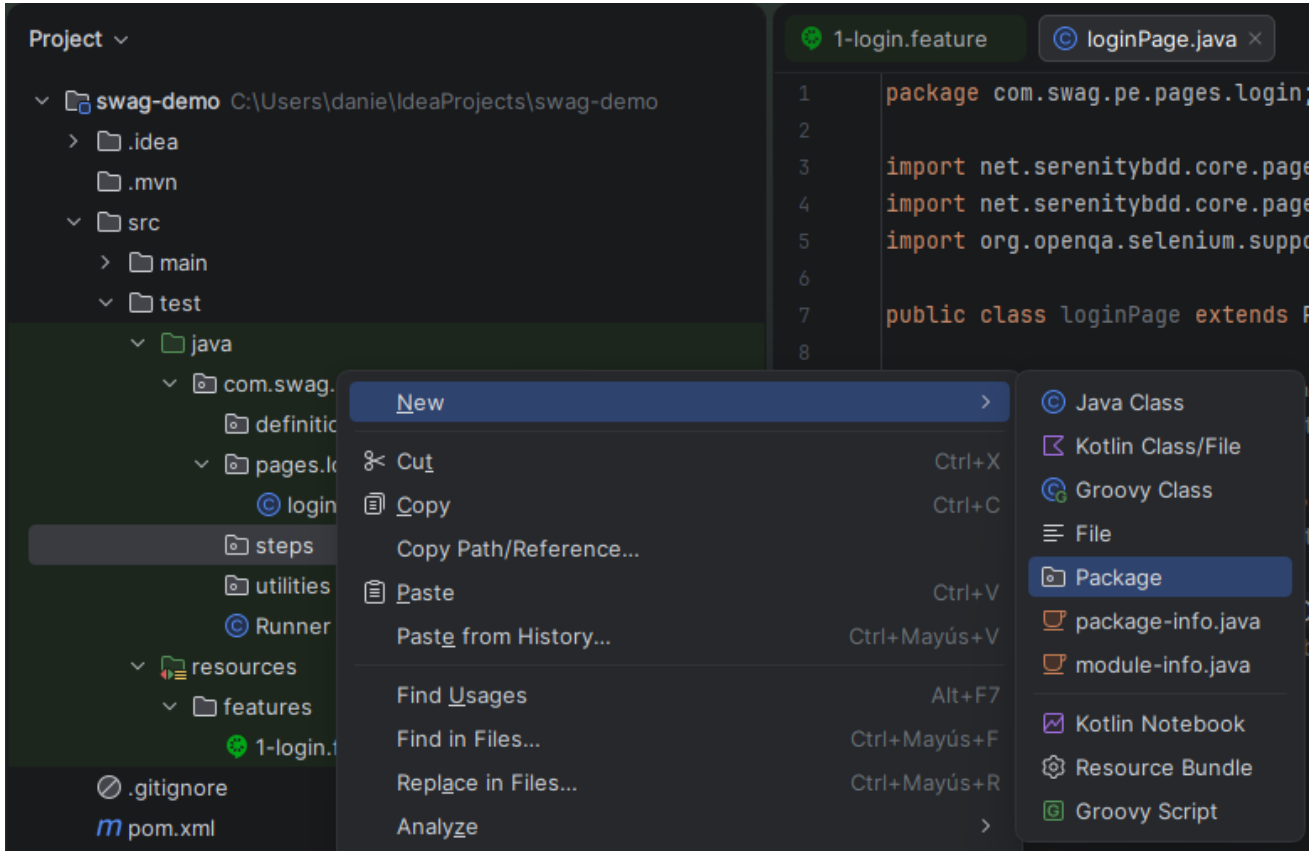
    @FindBy(id = "password")  no usages
    protected WebElementFacade txt_password;

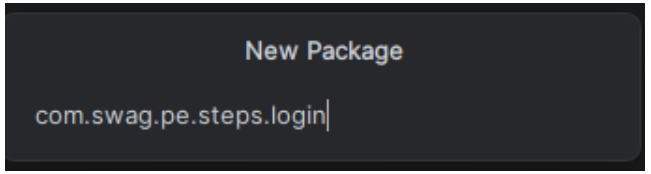
    @FindBy(id = "login-button")  no usages
    protected WebElementFacade btn_login;
}
```

idat

# Creamos el paquete Login en Steps

Dentro de com.swag.pe.steps, creamos el paquete: login

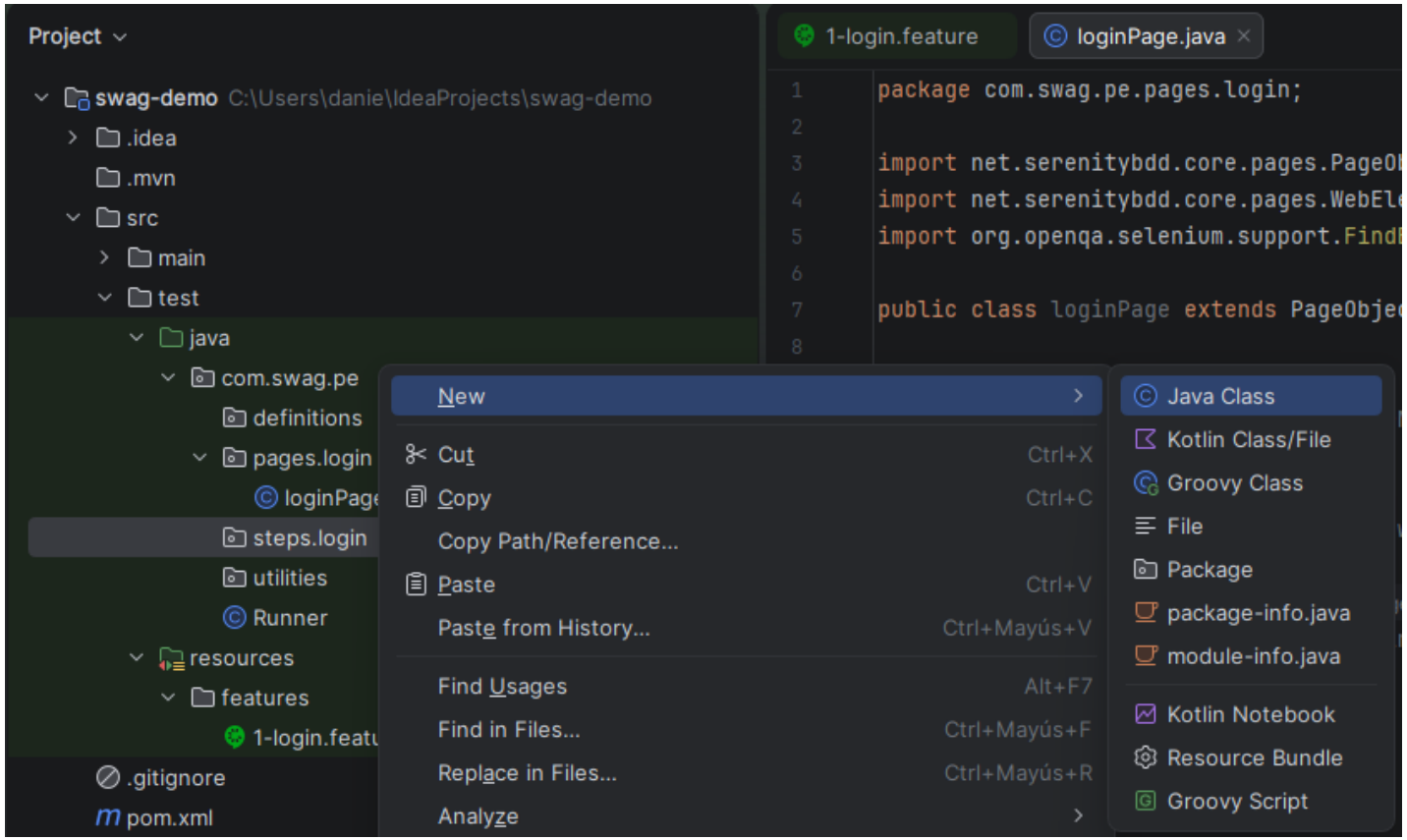


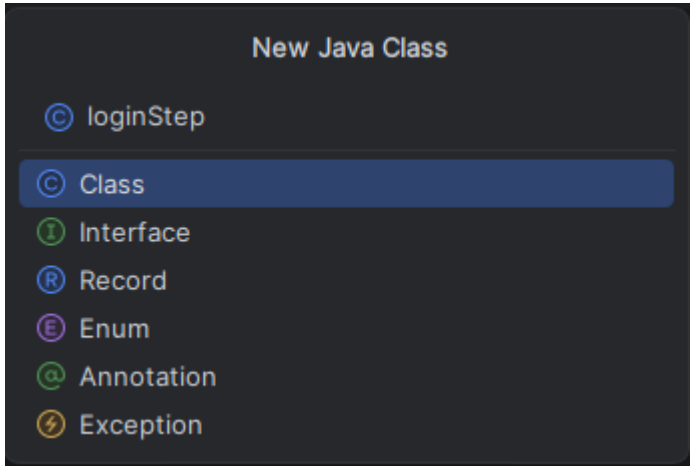




idat

# Creamos la clase LoginStep

Dentro de com.swag.pe.steps.login, creamos la clase: loginStep




idat

# Codificamos clase LoginStep

La extendemos de la clase loginPage que hemos creado

1-login.feature | loginPage.java | loginStep.java

```java
package com.swag.pe.steps.login;

import com.swag.pe.pages.login.loginPage;
import net.thucydides.core.annotations.Step;

public class loginStep extends loginPage {  no usages

    @Step("Ingresar Usuario")  no usages
    public void typeUserName(String userName) {
        txt_userName.sendKeys(userName);
    }

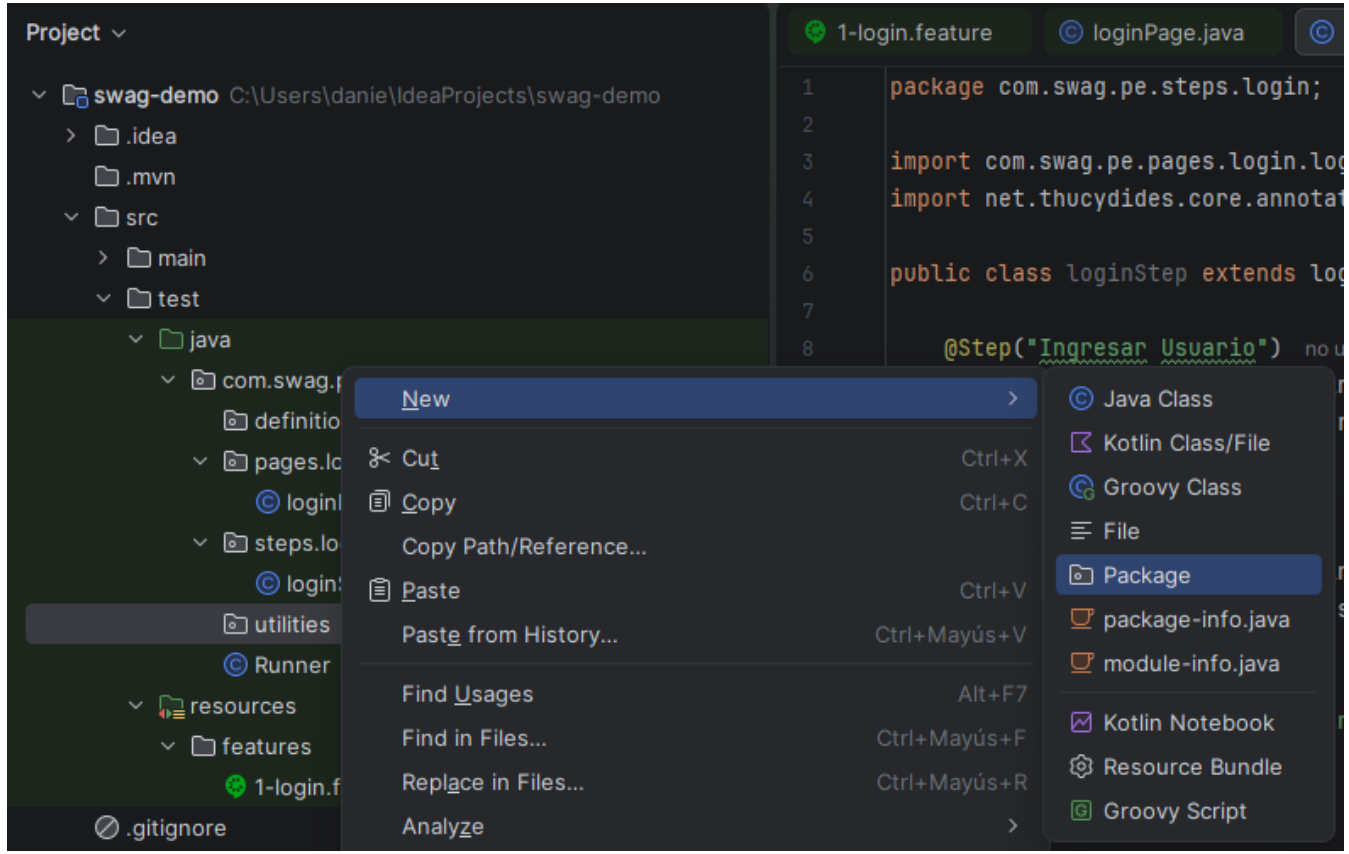
    @Step("Ingresar Contraseña")  no usages
    public void typePassword(String password) {
        txt_password.sendKeys(password);
    }

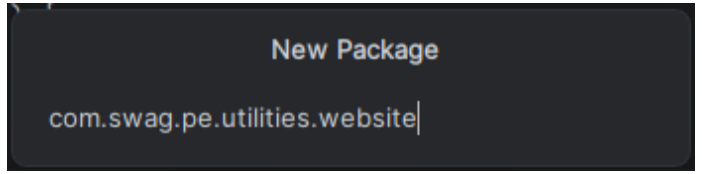
    @Step("Click en el botón Login")  no usages
    public void clickLogin() {
        btn_login.click();
    }
}
```

idat

# Creamos el paquete WebSite en Utilities

Dentro de com.swag.pe.utilities, creamos el paquete: website

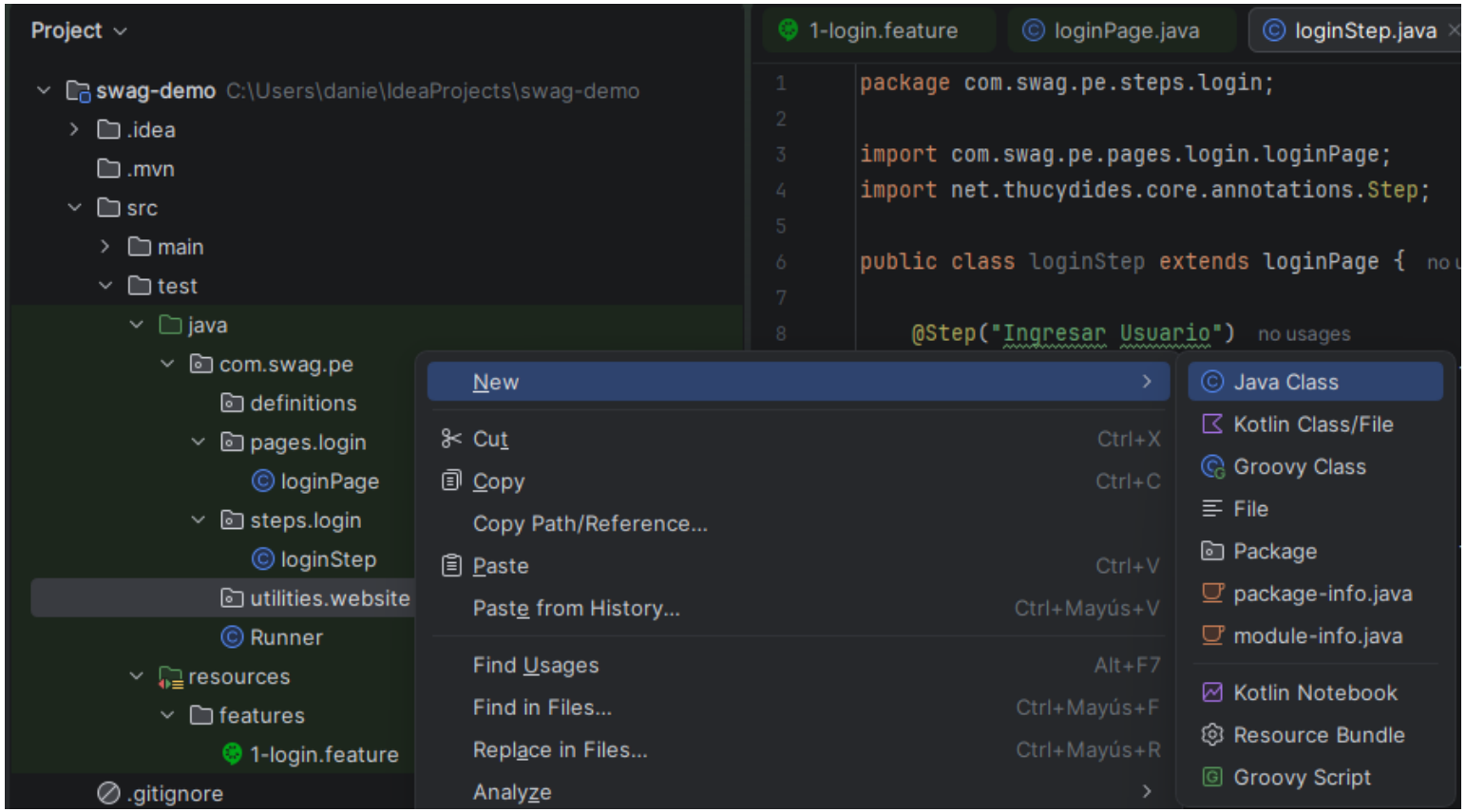


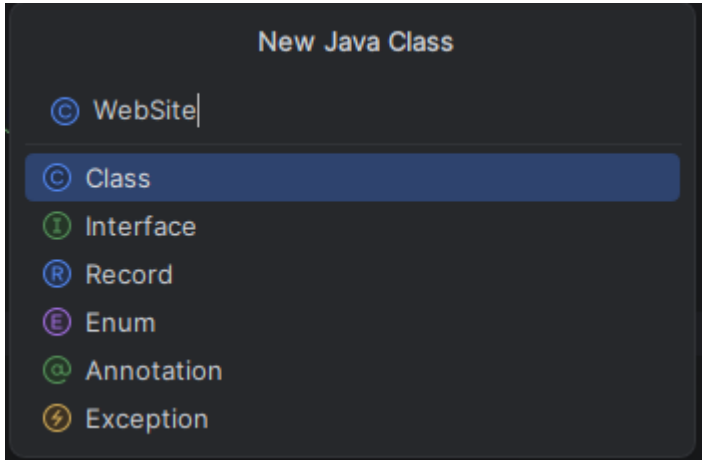




idat

# Creamos la clase WebSite

Dentro de com.swag.pe.utilities.website, creamos la clase: WebSite




idat

# Codificamos clase WebSite

1-login.feature | loginPage.java | loginStep.java | WebSite.java

```java
package com.swag.pe.utilities.website;

import net.serenitybdd.core.pages.PageObject;
import net.thucydides.core.annotations.Step;
import net.thucydides.core.annotations.Steps;

public class WebSite {  no usages

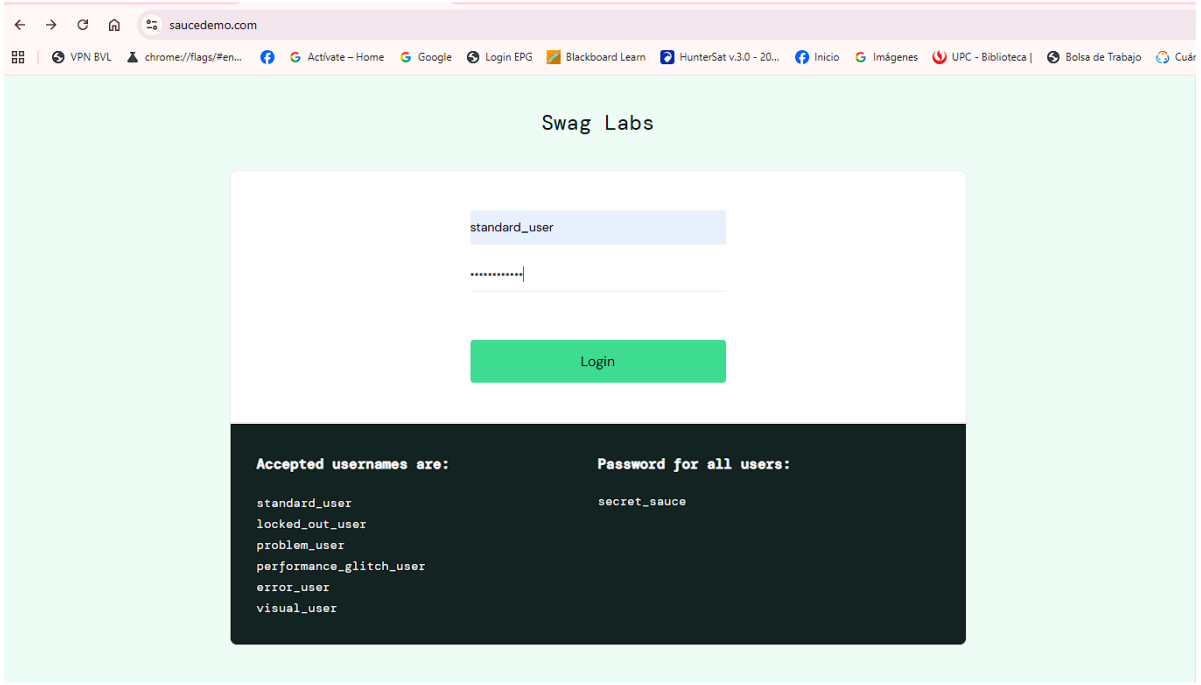
    @Steps(shared = true)  2 usages
    PageObject swag;

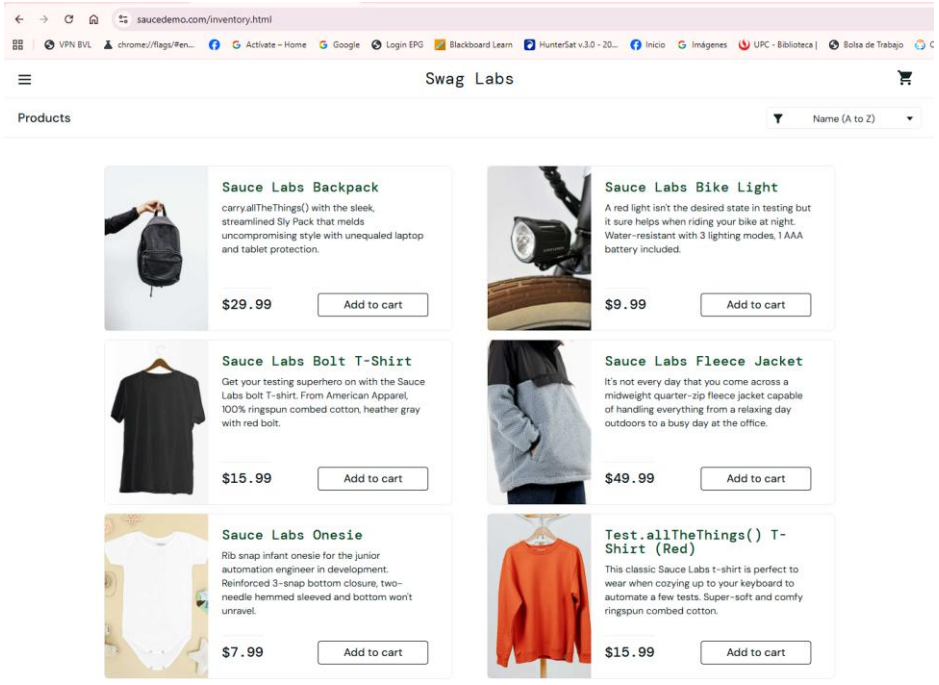
    @Step("Navegar al Sitio Web")  no usages
    public void navigateTo(String url) {
        swag.setDefaultBaseUrl(url);
        swag.open();
    }
}
```

idat

# Ingresamos a la Web

- Ingresamos al link https://www.saucedemo.com/
- Colocamos las credenciales correctas

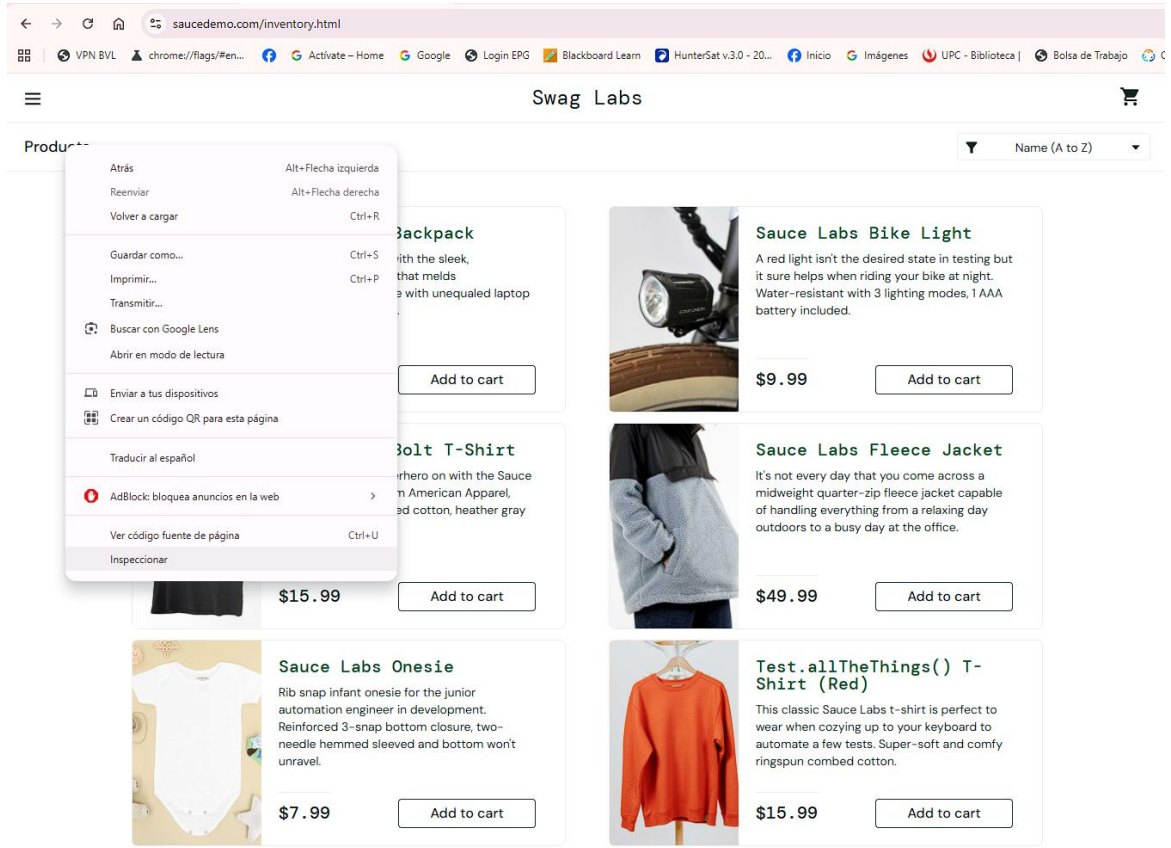


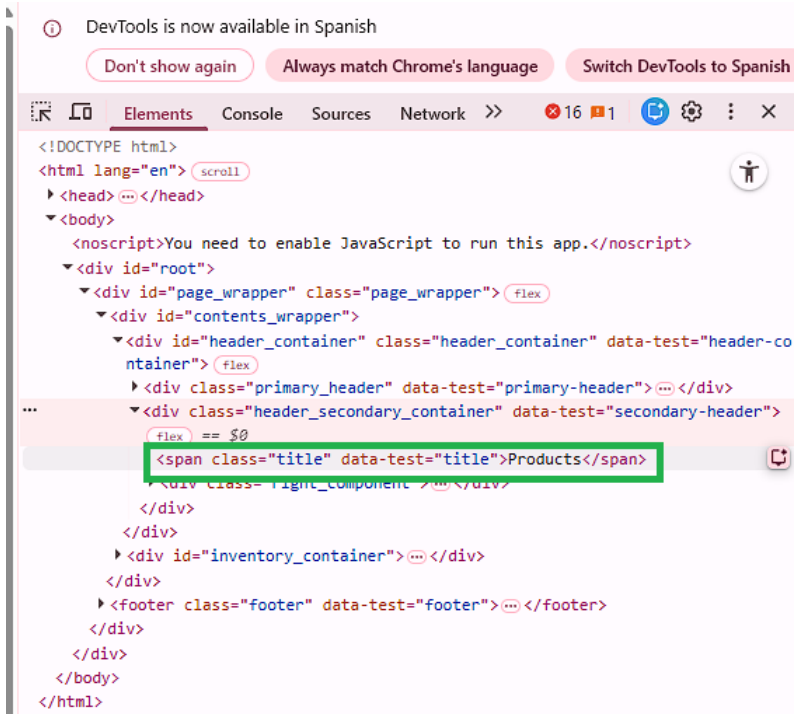




idat

# Validamos que ha cargado la página

- Hacemos click en “Products”, botón derecho y elegimos inspeccionar




idat

# Validamos que ha cargado la página

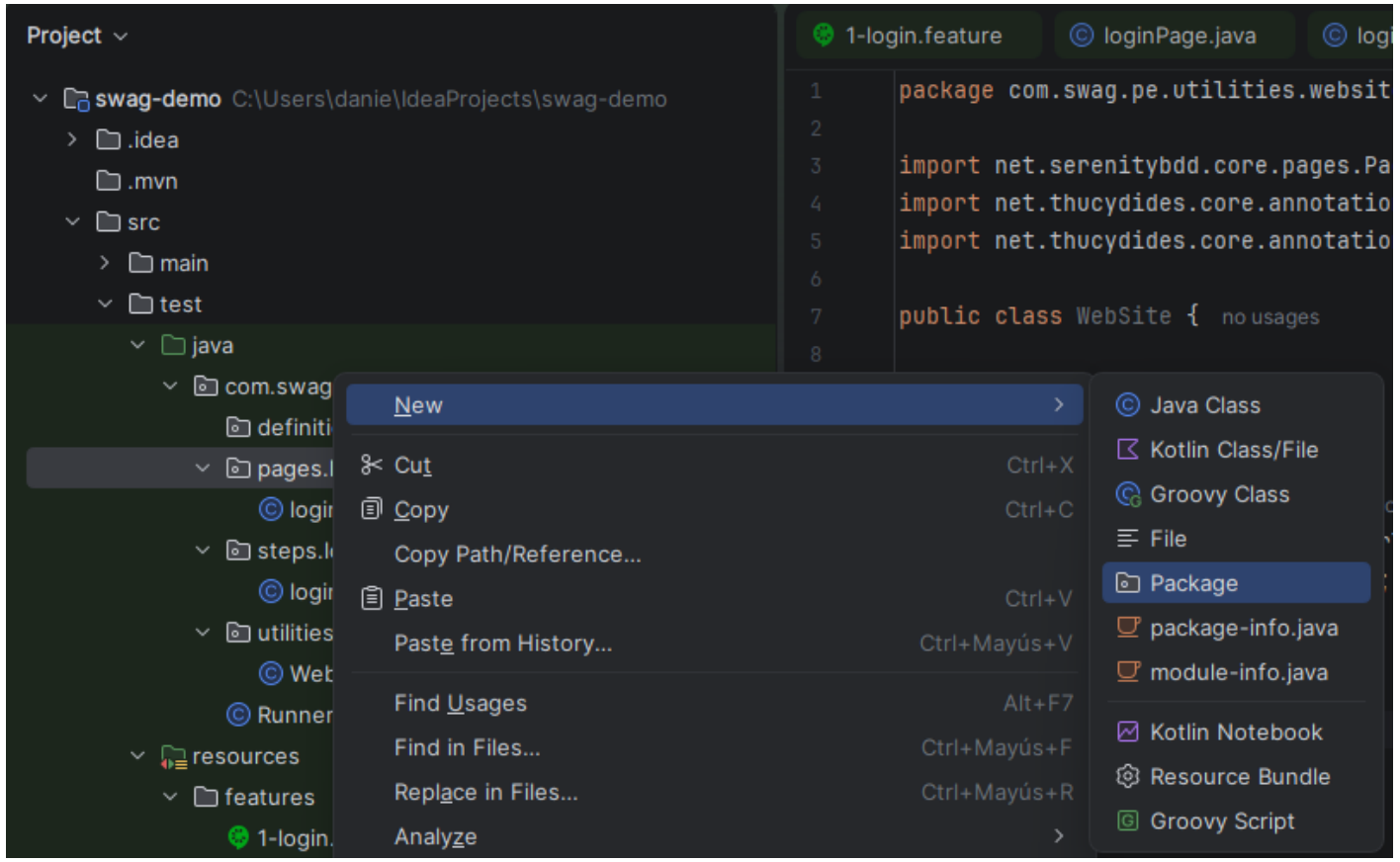
- Presionamos Ctrl-F en inspeccionar para validar la búsqueda
- En el cuadro de búsqueda escribimos:
  - //span[@class='title']

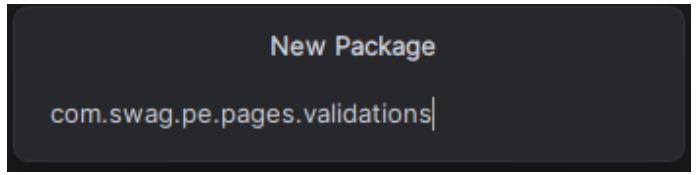



idat

# Creamos el paquete Validations en Pages

Dentro de com.swag.pe.pages, creamos el paquete: validations

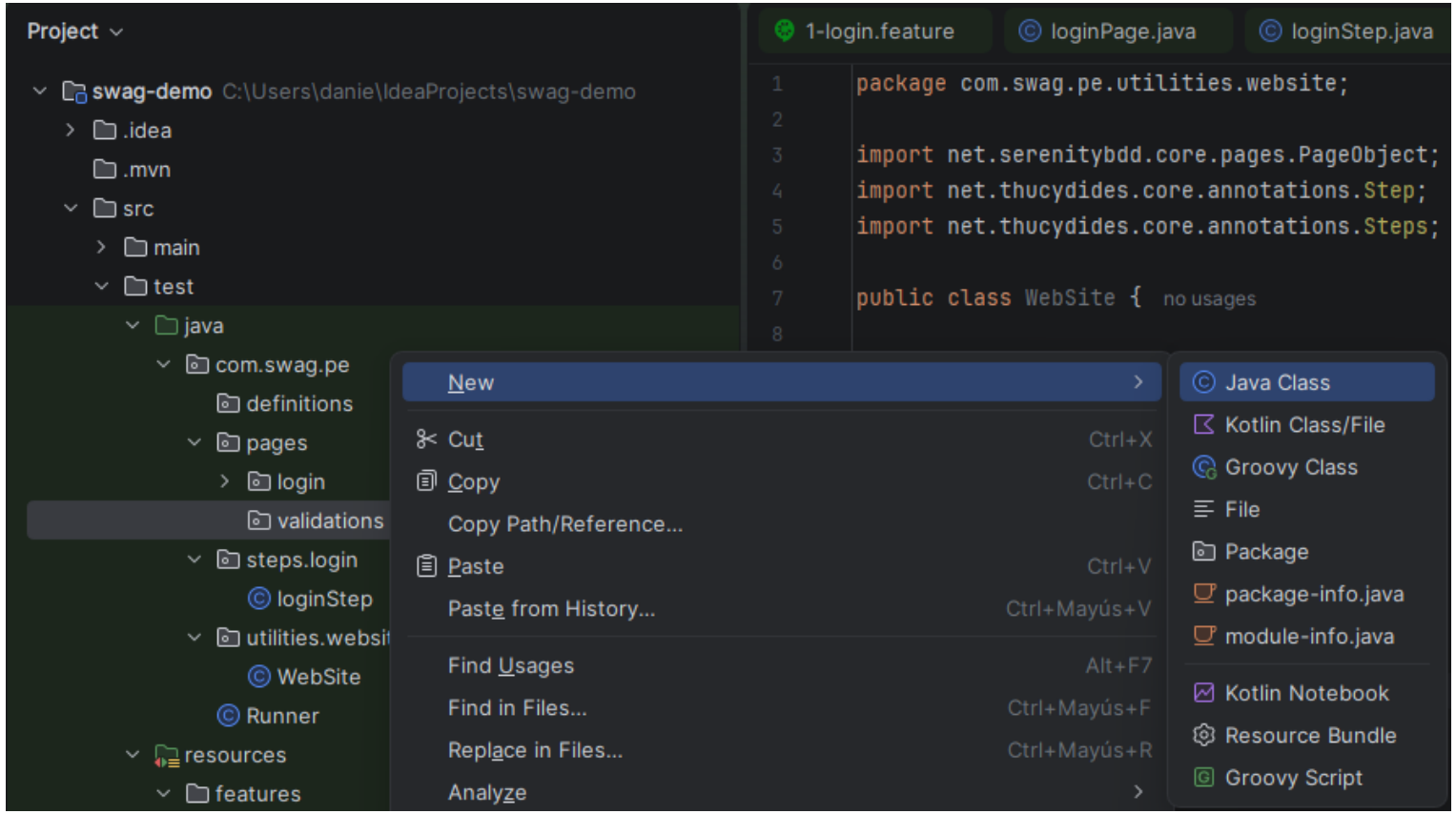


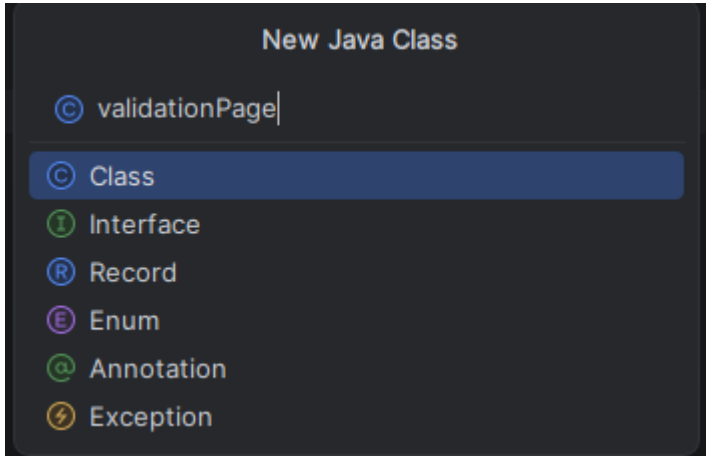




idat

# Creamos la clase ValidationPage

Dentro de com.swag.pe.pages.validations, creamos la clase: validationPage




idat

# Codificamos clase ValidationPage

La extendemos de la clase PageObject

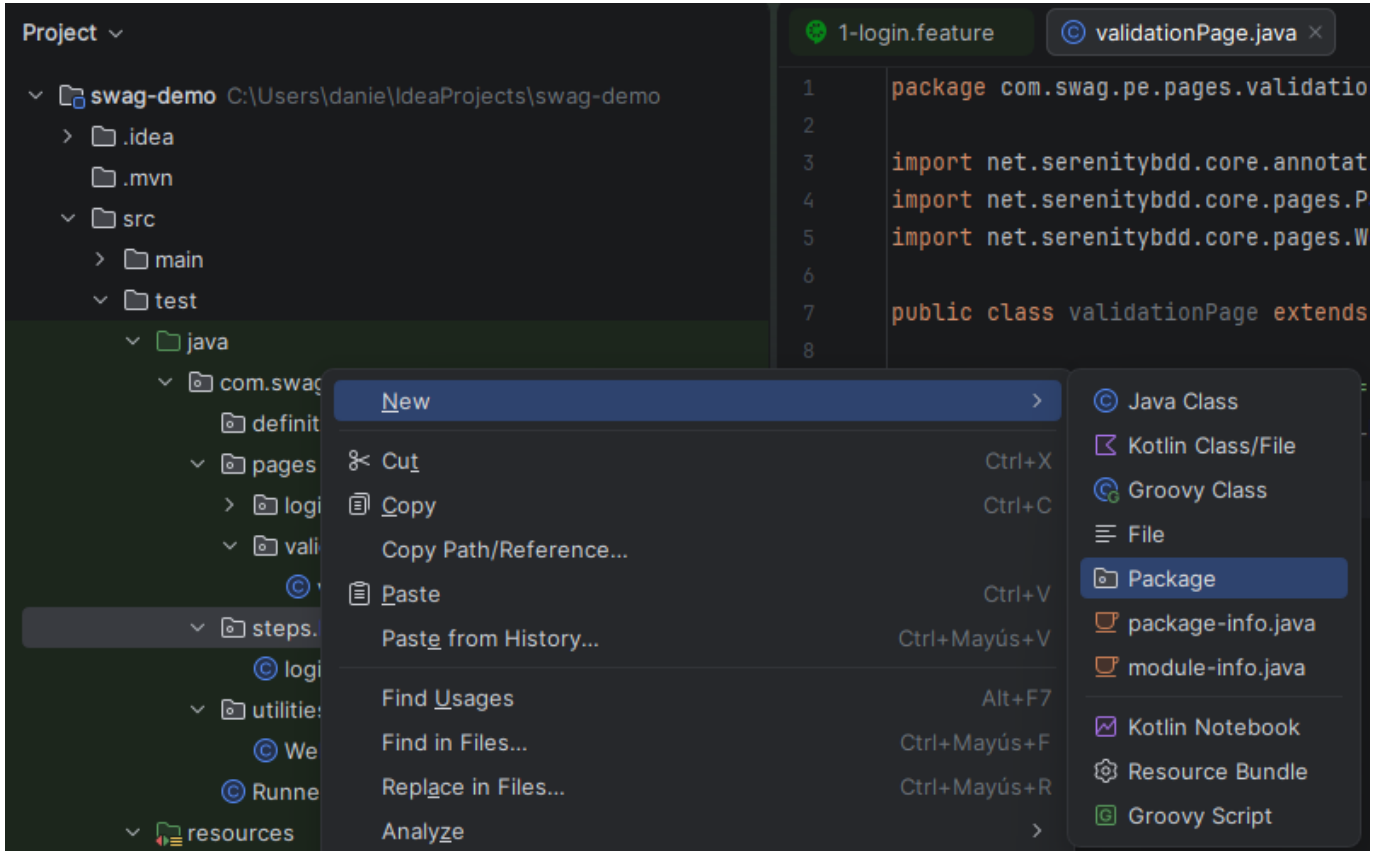
1-login.feature | validationPage.java

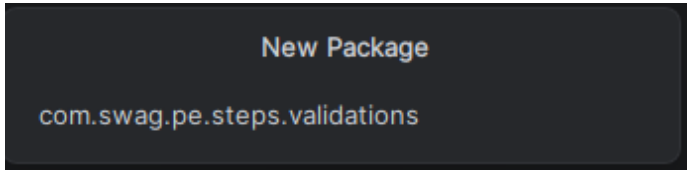
```
1   package com.swag.pe.pages.validations;
2
3   import net.serenitybdd.core.annotations.findby.FindBy;
4   import net.serenitybdd.core.pages.PageObject;
5   import net.serenitybdd.core.pages.WebElementFacade;
6
7   public class validationPage extends PageObject {  no usages
8
9       @FindBy(xpath = "//span[@class='title']")  no usages
10      protected WebElementFacade lbl_product;
11  }
```

idat

# Creamos el paquete Validations en Steps

Dentro de com.swag.pe.steps, creamos el paquete: validations

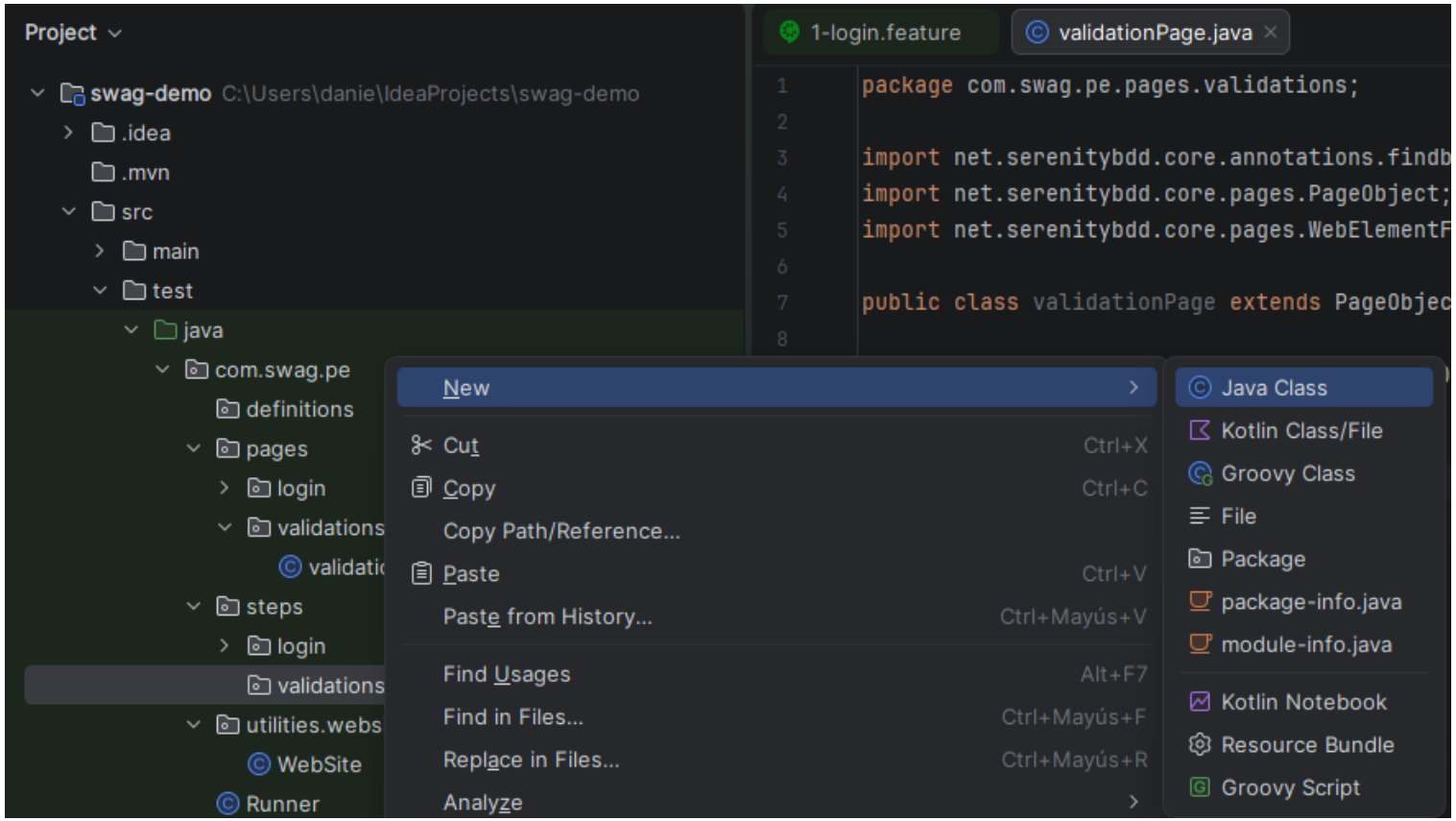


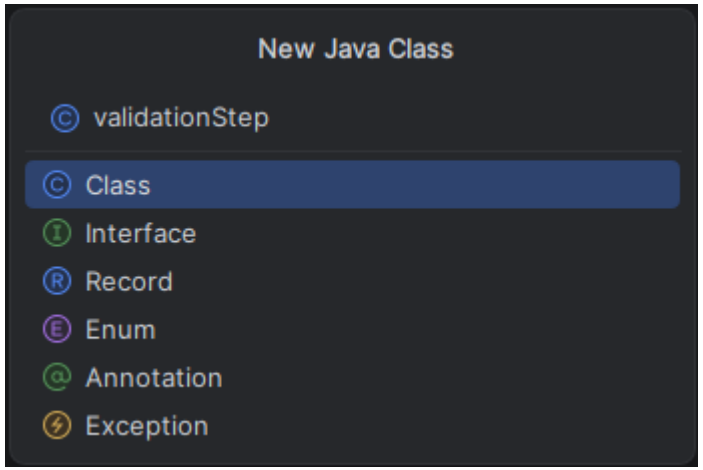




idat

# Creamos la clase ValidationStep

Dentro de com.swag.pe.steps.validations, creamos la clase: validationStep




idat

# Codificamos clase validationStep

La extendemos de la clase validationPage que hemos creado

1-login.feature | validationStep.java

```java
package com.swag.pe.steps.validations;

import com.swag.pe.pages.validations.validationPage;
import net.thucydides.core.annotations.Step;

public class validationStep extends validationPage {  no usages

    @Step("Validar visualizacion del modulo de productos")  no usages
    public boolean titleIsVisible() {
        return lbl_product.isDisplayed();
    }
}
```

idat

# Continuamos con las pruebas

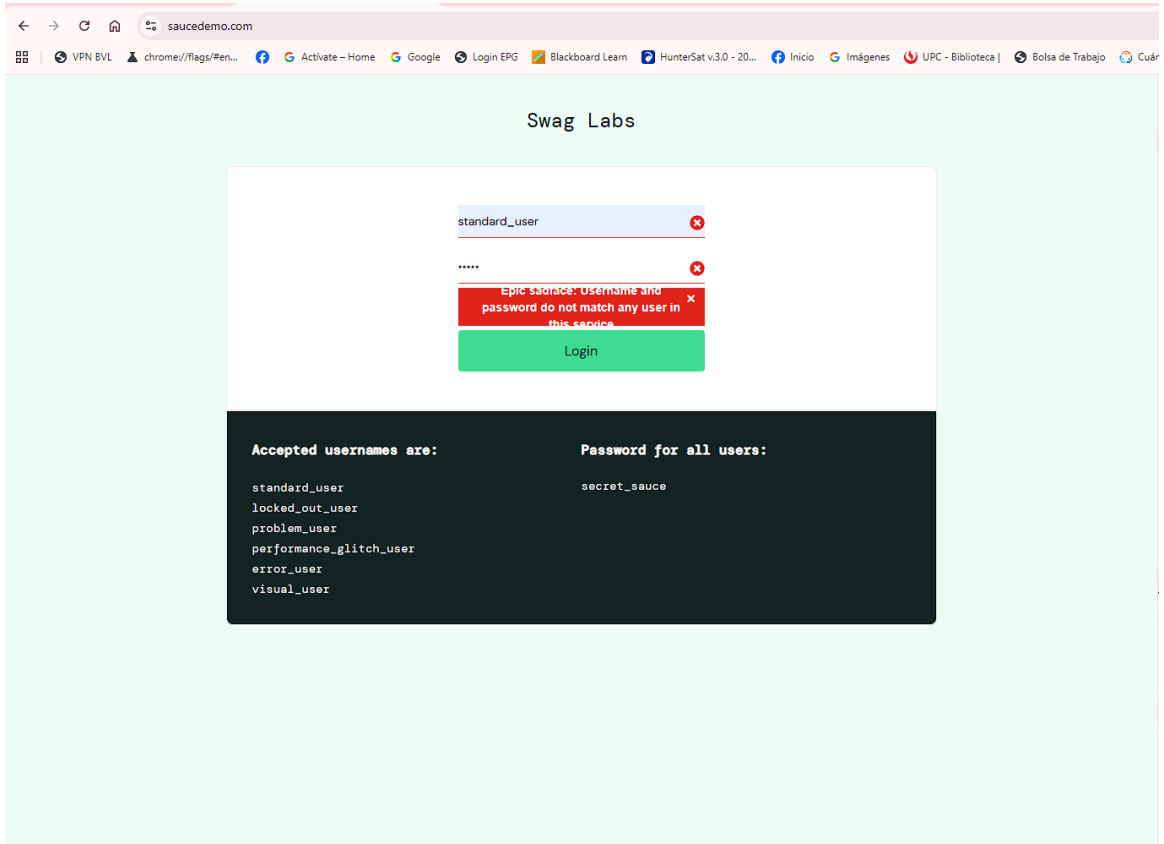
Hemos probado el camino feliz, ahora debemos probar que pasa si ingresamos la contraseña incorrecta




idat

# Ingresamos a la Web

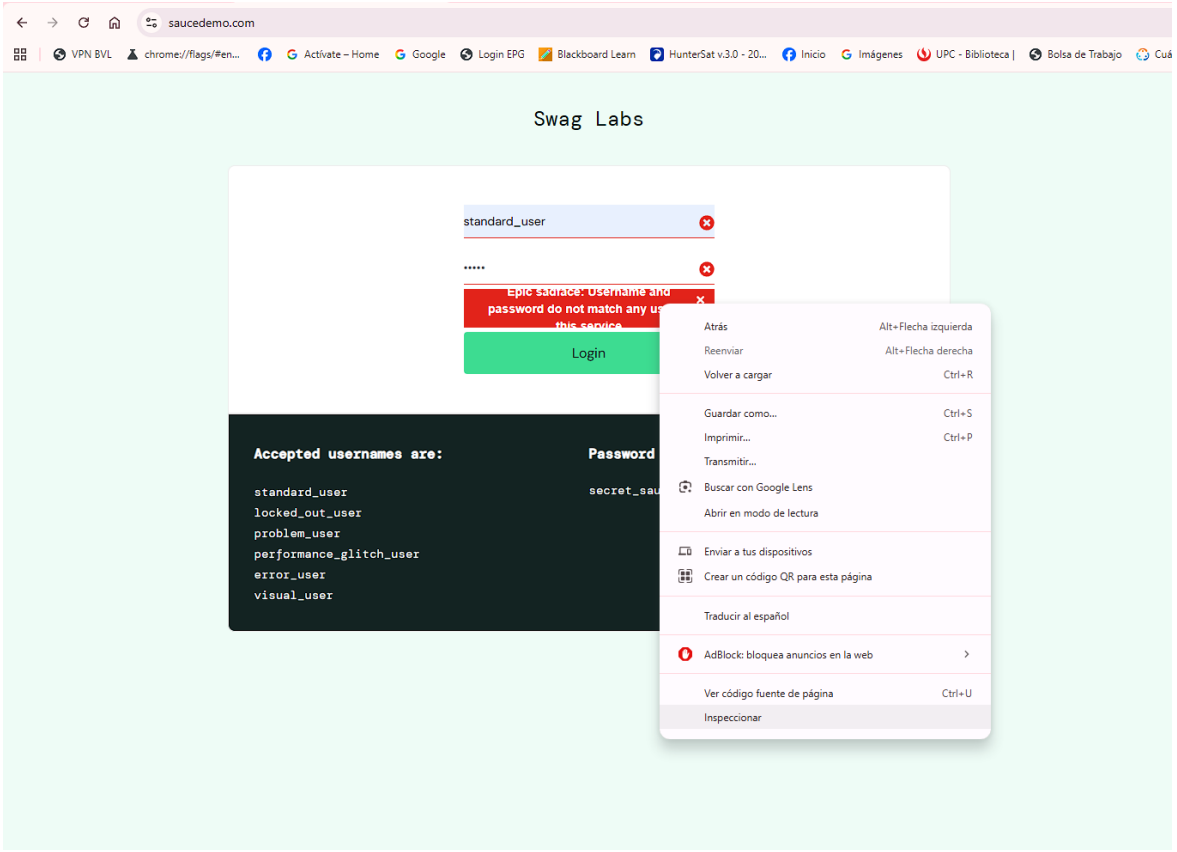
- Ingresamos al link https://www.saucedemo.com/
- Colocamos las credenciales incorrectas

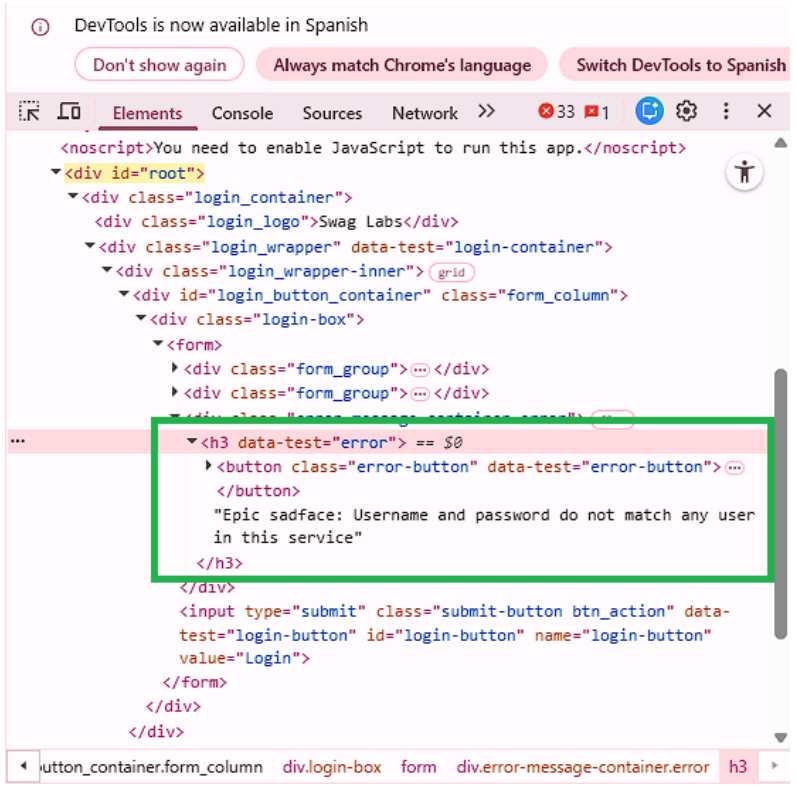



idat

# Validamos que ha salido error

- Hacemos click en el error, botón derecho y elegimos inspeccionar

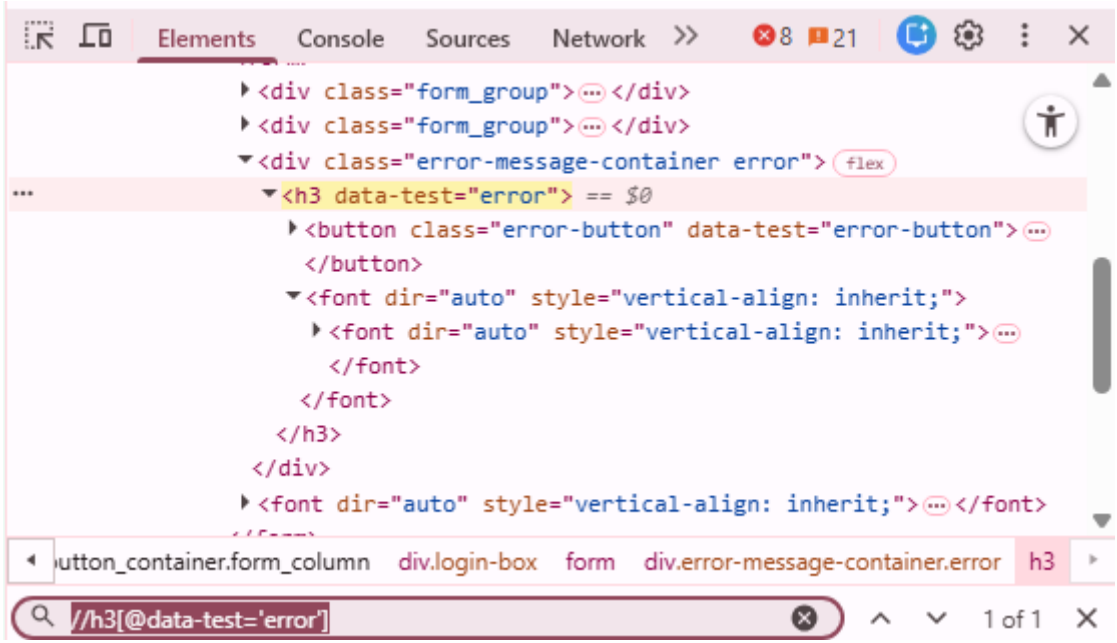



idat

# Validamos que ha salido error

- Presionamos Ctrl-F en inspeccionar para validar la búsqueda
- En el cuadro de búsqueda escribimos:
  - //h3[@data-test='error']




idat

# Continuamos con clase ValidationPage

Regresamos a la Clase validationPage  y creamos el Element para el login incorrecto

1-login.feature | validationPage.java | validationStep.java

```
1   package com.swag.pe.pages.validations;
2
3   import net.serenitybdd.core.annotations.findby.FindBy;
4   import net.serenitybdd.core.pages.PageObject;
5   import net.serenitybdd.core.pages.WebElementFacade;
6
7   public class validationPage extends PageObject {  2 usages  1 inheritor
8
9       @FindBy(xpath = "//span[@class='title']")  1 usage
10      protected WebElementFacade lbl_product;
11
12      @FindBy(xpath = "//h3[@data-test='error' and text()='Epic sadface: Username and password do not match any user in this service']")  no usages
13      protected WebElementFacade lbl_errorMessage;
14  }
```

idat

# Continuamos con clase validationStep

Función para el login incorrecto

1-login.feature | validationPage.java | validationStep.java

```java
package com.swag.pe.steps.validations;

import com.swag.pe.pages.validations.validationPage;
import net.thucydides.core.annotations.Step;

public class validationStep extends validationPage {  no usages

    @Step("Validar visualización del módulo de productos")  no usages
    public boolean tittleIsVisible() {
        return lbl_product.isDisplayed();
    }

    @Step("Validar visualización del mensaje de error")  no usages
    public boolean errorMessage() {
        return lbl_errorMessage.isDisplayed();
    }
}
```

idat

# Primer Caso Completo

Hasta aquí hemos completado el Primer Caso:

- Prueba con Login Correcto
- Prueba con Login Incorrecto

Pero…

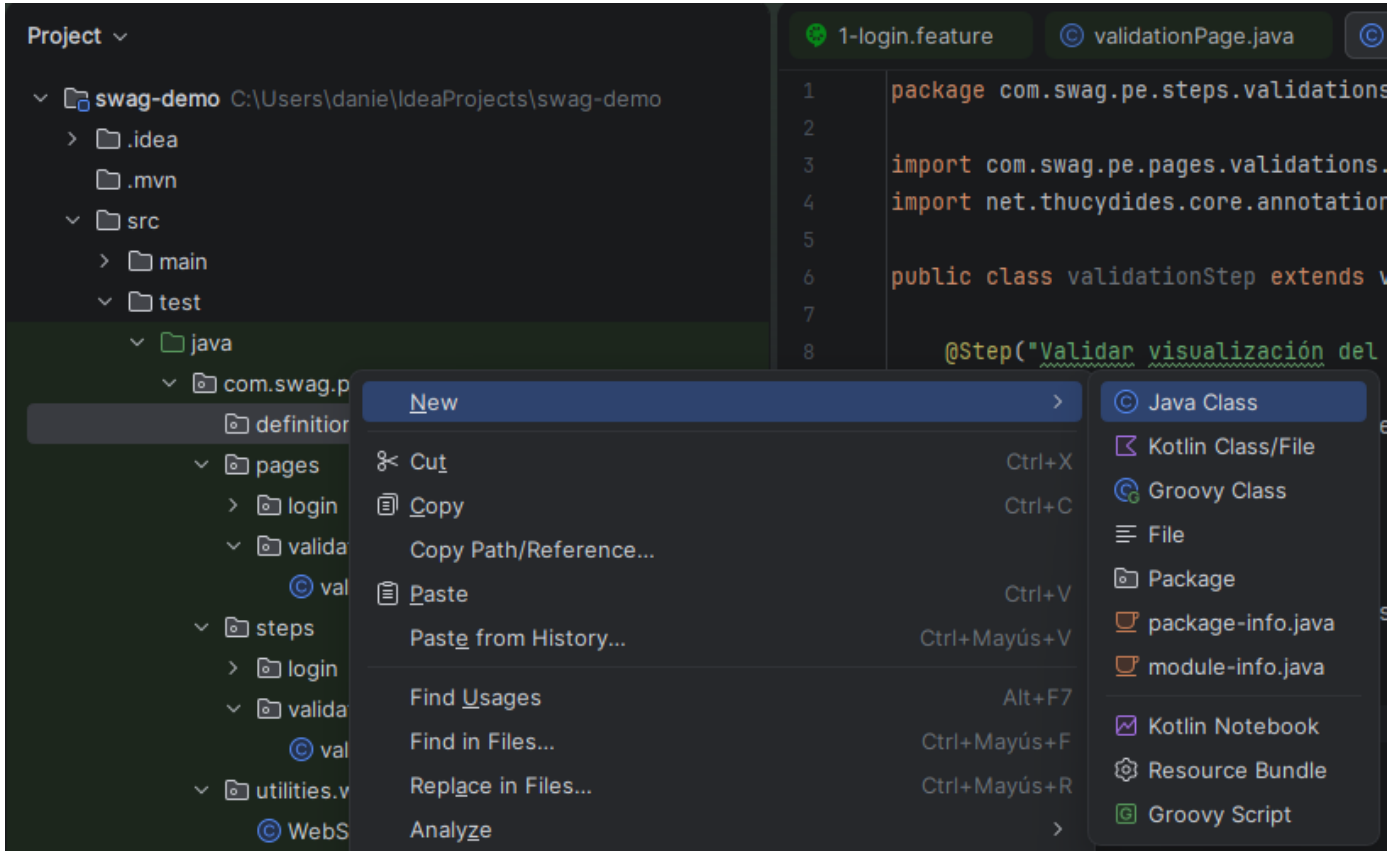
Falta configurar los Steps Definitions

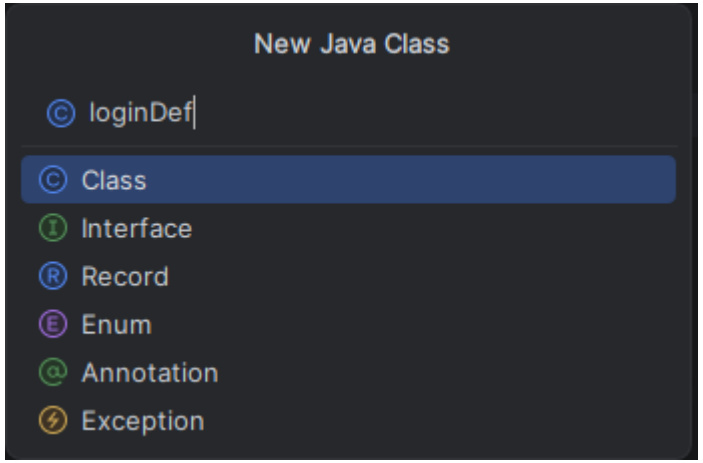


idat

# Creamos la clase LoginDef

Dentro de com.swag.pe.definitions, creamos la clase: loginDef

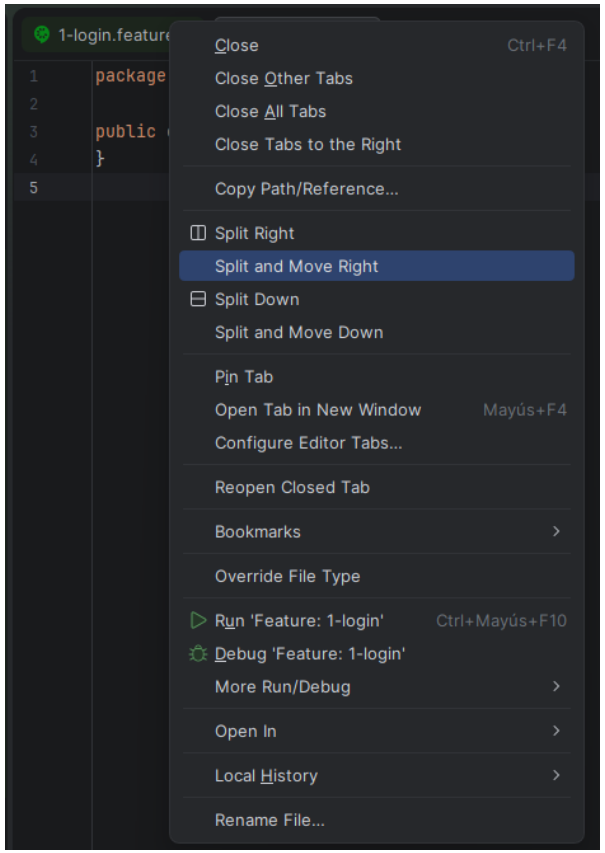


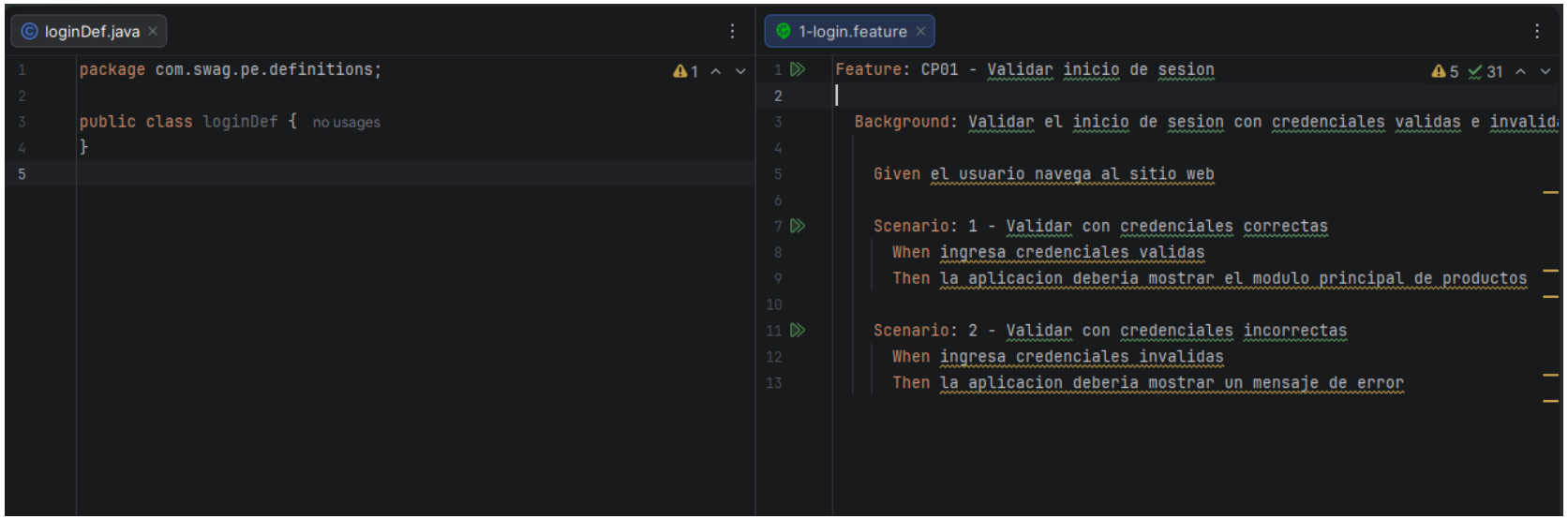




idat

# Creamos la clase LoginDef

Hacemos click en la parte superior de “1-login.features” y click derecho, seleccionamos “Split and move right”




idat

# Codificamos clase loginDef

```java
public class loginDef {

    @Steps(shared = true)  1 usage
    WebSite url;

    @Steps(shared = true)  6 usages
    loginStep login;

    @Steps(shared = true)  2 usages
    validationStep validate;
```

```java
    @Given("el usuario navega al sitio web")
    public void userNavigateTo() {
        url.navigateTo( url: "https://www.saucedemo.com/");
    }

    @When("ingresa credenciales validas")
    public void userLoginWithValidCredentials() {
        login.typeUserName("standard_user");
        login.typePassword("secret_sauce");
        login.clickLogin();
    }

    @Then("la aplicacion deberia mostrar el modulo principal de product
    public void systemShowProducts() {
        Assert.assertTrue(validate.tittleIsVisible());
    }

    @When("ingresa credenciales invalidas")
    public void userLoginWithInvalidCredentials() {
        login.typeUserName("standard_user");
        login.typePassword("12345");
        login.clickLogin();
    }

    @Then("la aplicacion deberia mostrar un mensaje de error")
    public void systemShowErrorMessage() {
        Assert.assertTrue(validate.errorMessage());
    }
```

idat

# Colocamos “tags” al Feature: login

Ponerle “tags” me permite indicarle a Serenity que quiero probar, si tengo muchos escenarios

loginDef.java | Runner.java | 1-login.feature

```
@Suite @Login
Feature: CP01 - Validar inicio de sesion

  Background: Validar el inicio de sesion con credenciales validas e invalidas

    Given el usuario navega al sitio web

    @ValidCredentials
    Scenario: 1 - Validar con credenciales correctas
      When ingresa credenciales validas
      Then la aplicacion deberia mostrar el modulo principal de productos

    @InvalidCredentials
    Scenario: 2 - Validar con credenciales incorrectas
      When ingresa credenciales invalidas
      Then la aplicacion deberia mostrar un mensaje de error
```

idat

# Codificamos la Clase: Runner

Runner.java

```java
package com.swag.pe;

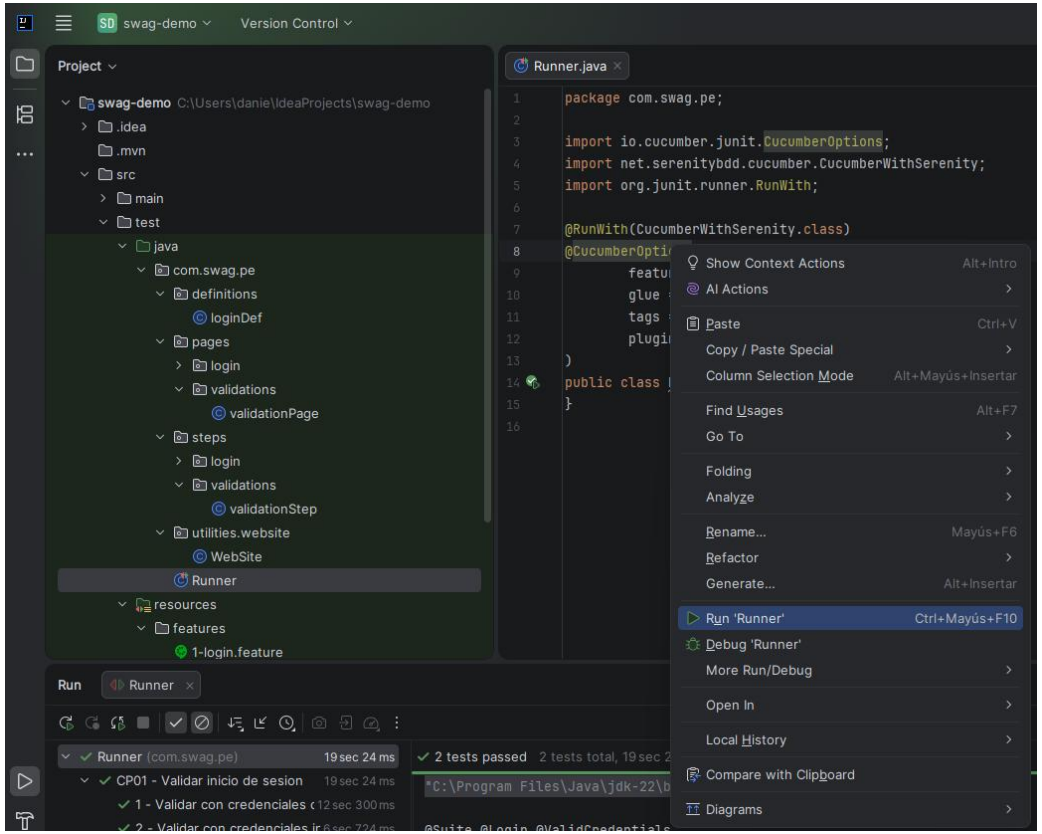
import io.cucumber.junit.CucumberOptions;
import net.serenitybdd.cucumber.CucumberWithSerenity;
import org.junit.runner.RunWith;

@RunWith(CucumberWithSerenity.class)
@CucumberOptions(
        features = "src/test/resources/features",
        glue = "com.swag.pe.definitions",
        tags = "@Login",
        plugin = "pretty"
)
public class Runner {
}
```

idat

# Ejecutamos nuestra prueba

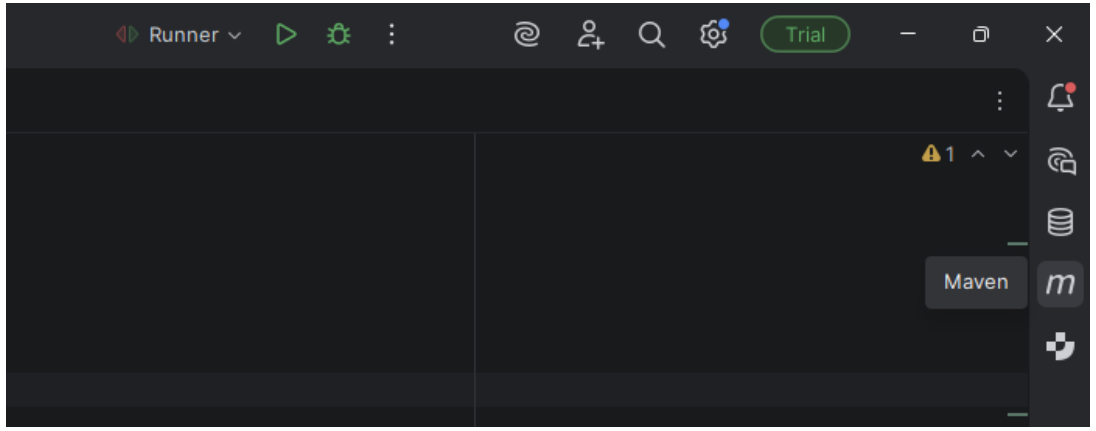
Click derecho y seleccionar “Run Runner”

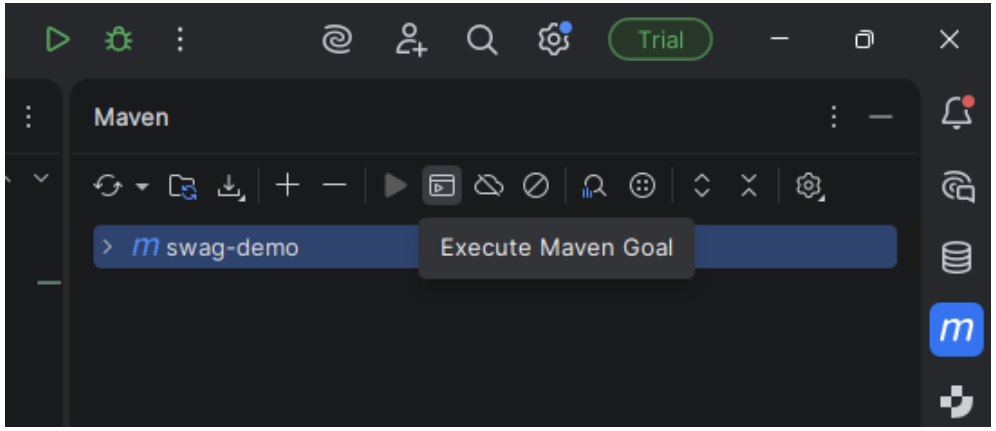



idat

# Ver el reporte de Serenity

- Seleccionamos “Maven” en el menú del lado derecho
- Seleccionamos “Ejecutar Maven Goal”



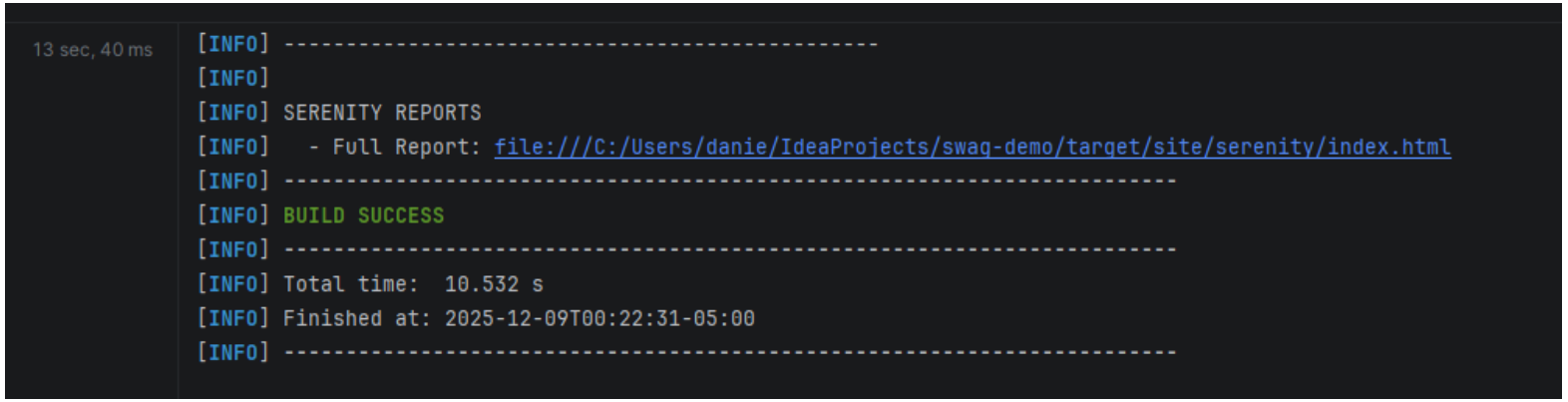




idat

# Ver el reporte de Serenity

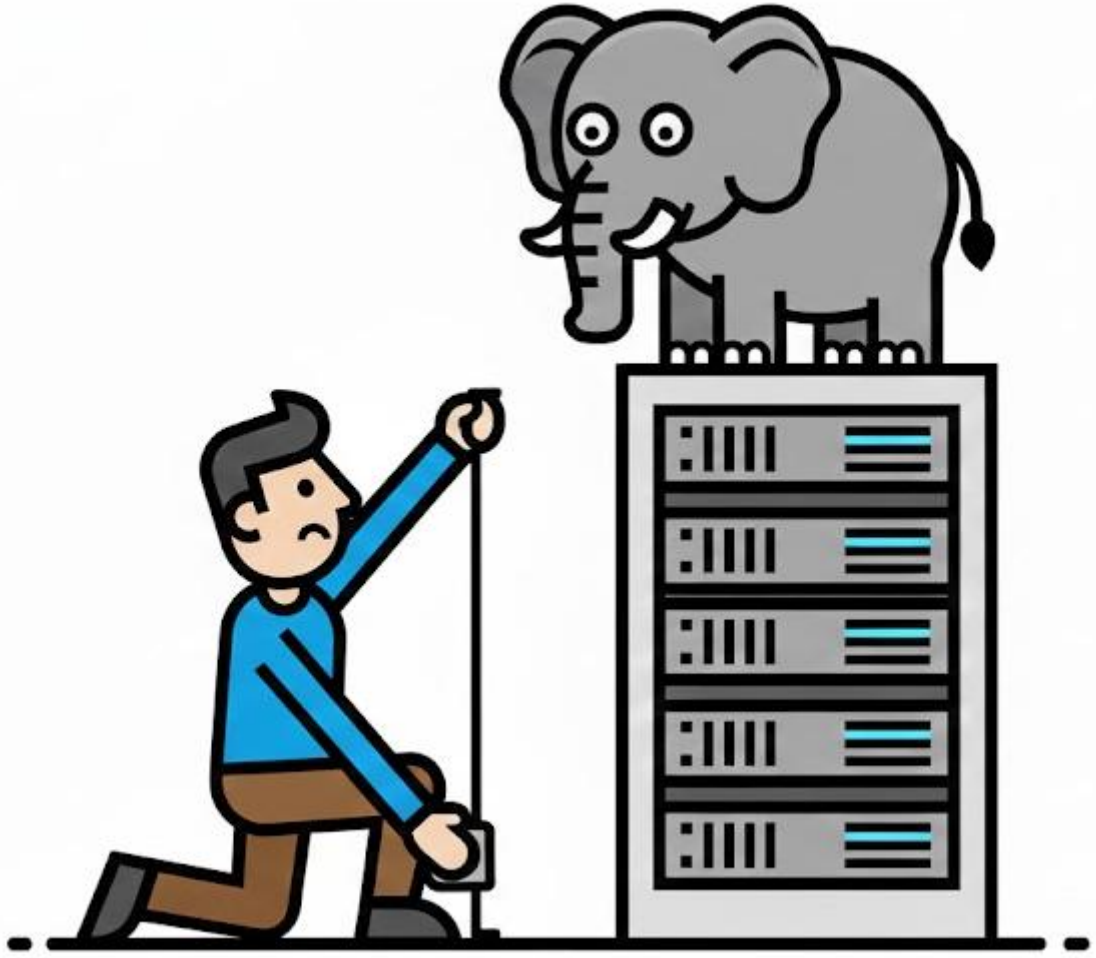
- Luego: mvn serenity:aggregate
- Luego, click en el reporte




idat

# Pruebas de rendimiento con JMeter



idat

# Introducción

El mundo digital exige que las aplicaciones gestionen estándares de rendimiento en constante aumento. Los usuarios exigen interacciones fluidas e inmediatas con las aplicaciones en cualquier condición de tráfico y durante las complejas secuencias de procesos. Su software debe afrontar estos desafíos para garantizar la satisfacción del usuario y el éxito empresarial.

Implementar pruebas de rendimiento es esencial, ya que permite simular un tráfico de usuarios realista, lo que permite identificar cuellos de botella en el rendimiento, evaluar la escalabilidad y predecir las reacciones del sistema en diferentes condiciones. La información obtenida ayuda a optimizar la aplicación con antelación para ofrecer a los usuarios experiencias consistentes y de alta calidad en todos los escenarios.

# ¿Qué es JMeter?

JMeter es una aplicación Java de código abierto para realizar pruebas de rendimiento y carga en aplicaciones web, API y otros servicios. Este software se utiliza para probar aplicaciones web, revisar las API y cargar otros servicios.

Los desarrolladores comenzaron con el objetivo de ofrecer la herramienta para probar aplicaciones web, pero el proyecto se ha expandido y también acomoda aplicaciones como FTP, bases de datos y servidores TCP.

En resumen, es una aplicación que imita a múltiples usuarios que realizan solicitudes en un servidor de destino, registrando los tiempos de respuesta, el rendimiento y todos los demás factores de rendimiento vitales.



APACHE JMeter™

idat

# Características principales de JMeter

- **Código abierto:** JMeter es gratuito y tiene una comunidad activa que apoya su desarrollo y mejoras.
- **Independiente de la plataforma:** escrito en Java, se ejecuta en múltiples sistemas operativos como Windows, macOS y Linux.
- **GUI fácil de usar:** proporciona una interfaz gráfica intuitiva que simplifica la creación y ejecución de planes de prueba.
- **Compatibilidad con múltiples protocolos:** admite HTTP, HTTPS, FTP, JDBC, SOAP, REST, JMS y muchos más.
- **Extensible:** es una disposición para la integración del complemento para ampliar la funcionalidad.
- **Grabación de script de prueba:** el usuario puede grabar fácilmente sus acciones para crear un caso de prueba.

idat

# Características principales de JMeter

- **Informes completos:** son conocidos por sus informes estadísticos y representaciones gráficas para el análisis de datos.
- **Simulación de usuarios concurrentes:** utilice esta función para simular el comportamiento de miles de usuarios y ver cómo funcionará el sistema bajo cargas elevadas.
- **Automatización e integración:** la aplicación está automatizada y se puede activar desde la línea de comandos o una canalización CI/CD.

# ¿Qué son las pruebas de rendimiento?

Las pruebas de rendimiento de JMeter consisten en evaluar la velocidad, la capacidad de respuesta y la fiabilidad de una aplicación bajo diferentes niveles de carga. Identifican cuellos de botella en el rendimiento antes de implementar una aplicación en producción. Con JMeter, los evaluadores pueden simular usuarios virtuales enviando solicitudes a servidores y recopilar métricas como:

- Tiempo de respuesta
- Rendimiento
- Tasas de error
- Utilización de recursos del servidor

# ¿Qué son las pruebas de rendimiento?

**JMeter puede realizar los siguientes tipos de pruebas de rendimiento:**

- **Prueba de carga:** muestra el comportamiento del sistema bajo una carga esperada de usuarios.
- **Pruebas de estrés:** así es como se pueden detectar los cuellos de botella en el sistema aumentando considerablemente la carga.
- **Prueba de picos:** significa probar si el rendimiento de una carga es bueno cuando se implementa instantáneamente o no.
- **Prueba de resistencia:** este método verificará cuidadosamente la estabilidad de su sistema cuando active la carga durante un período más prolongado.
- **Prueba de escalabilidad:** verifica qué tan bien responde el sistema cuando crece el número de usuarios.

idat

# Guía paso a paso para realizar pruebas de rendimiento con JMeter

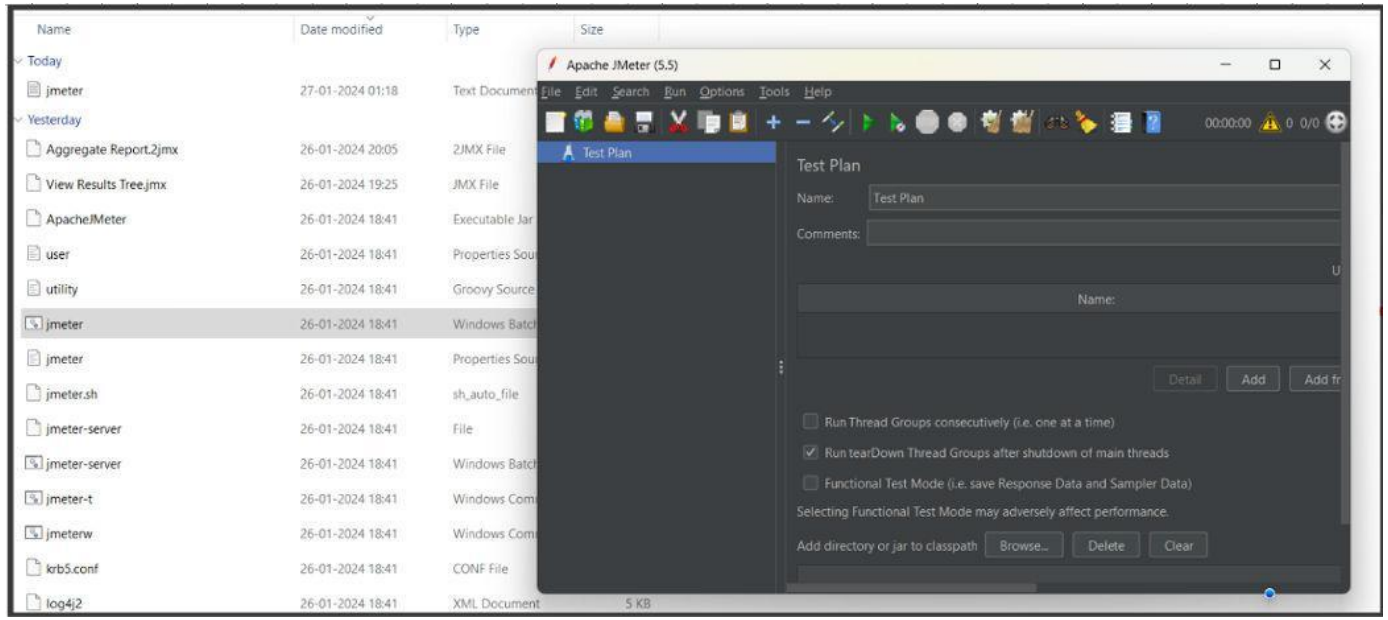



idat

# Guía paso a paso

## Paso 1: Instalar JMeter

1. Descargue JMeter desde el **sitio web oficial** (https://jmeter.apache.org/download_jmeter.cgi)
2. Extraiga el archivo descargado a la ubicación deseada.
3. Asegúrese de que **Java Development Kit (JDK)** esté instalado (JMeter requiere Java para ejecutarse).
4. Abra la carpeta bin y ejecute jmeter.bat (Windows) o jmeter.sh (Linux/macOS) para iniciar JMeter.

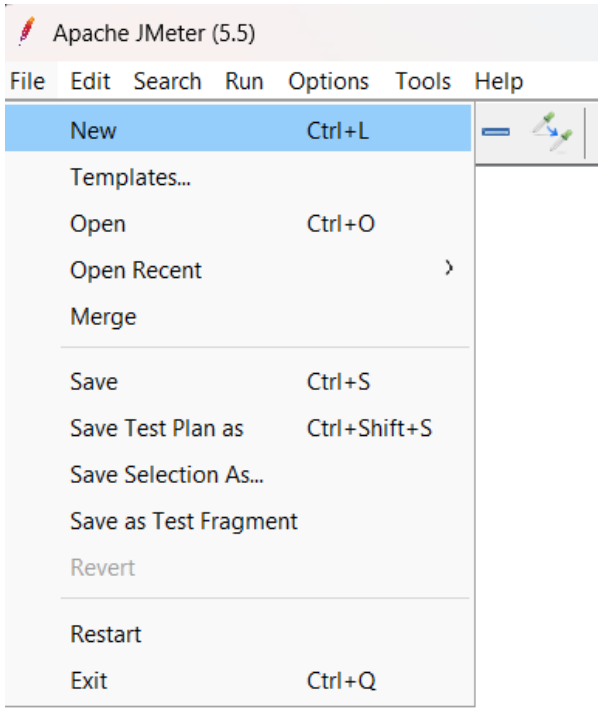



idat

# Guía paso a paso

## Paso 2: Crear un plan de pruebas

1. Abra JMeter y cree un **plan de prueba** (Archivo → Nuevo).
2. Haga clic con el botón derecho en el **plan de prueba** y agregue un **grupo de subprocesos** (Agregar → Subprocesos (Usuarios) → Grupo de subprocesos).
    - Establecer el **número de subprocesos (usuarios)**.
    - Definir el **período de aumento gradual** (tiempo para alcanzar el número total de usuarios).
    - Establezca el **número de bucles** (número de iteraciones por usuario).



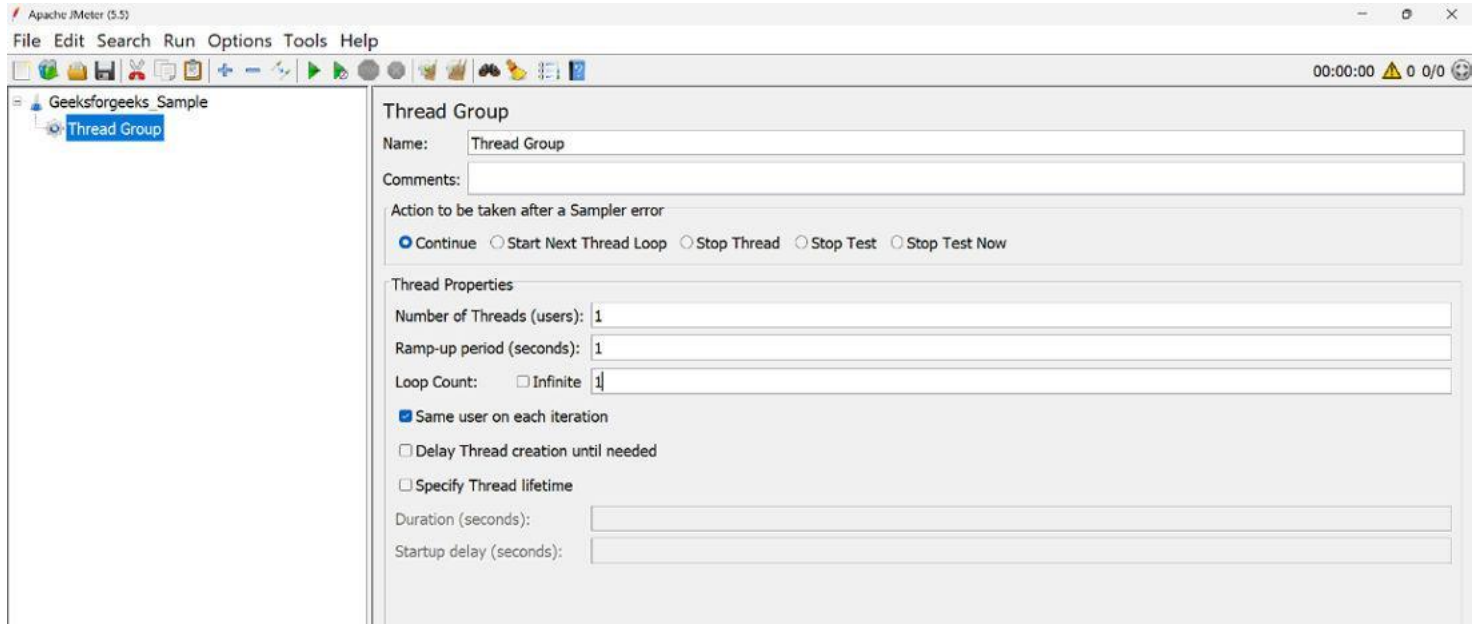

idat

# Guía paso a paso

## Paso 3: Agregar un grupo de hilos

1. Haga clic derecho en el **plan de prueba** en el panel izquierdo.
2. Seleccione **Agregar** → **Hilos (Usuarios)** → **Grupo de hilos** en el menú.
3. Establezca la cantidad de usuarios (subprocesos), la velocidad a la que comienzan (período de aumento gradual) y cuántas veces debe repetirse la prueba (cantidad de bucles).

Estos ajustes ayudan a simular la actividad real del usuario para las pruebas de rendimiento. Ajústelos según sus necesidades específicas.

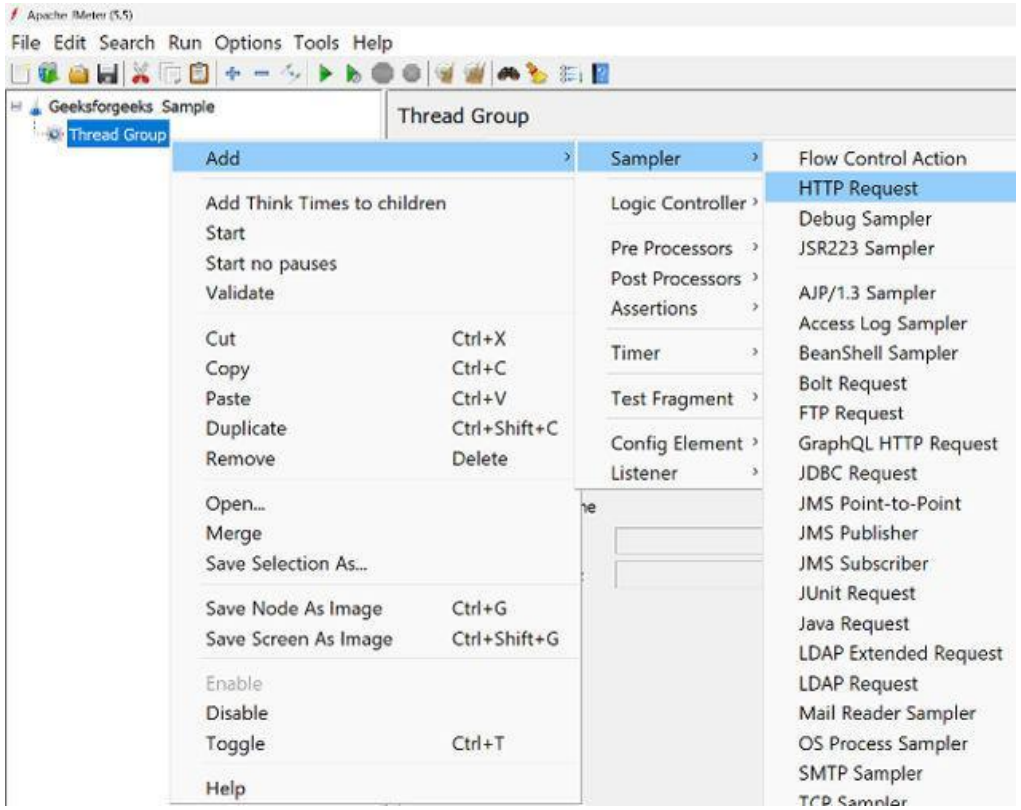



idat

# Guía paso a paso

## Paso 4: Agregar un sampler (solicitud HTTP)

1. Haga clic derecho en el **grupo de subprocesos** y agregue un muestreador HTTP (Agregar → Muestreador → Solicitud HTTP).
2. Introduzca el **nombre del servidor o la IP** (por ejemplo, www.ejemplo.com).
3. Establezca la **ruta de solicitud** (por ejemplo, /login para probar la página de inicio de sesión).
4. Defina métodos HTTP (GET, POST) y parámetros según sea necesario.

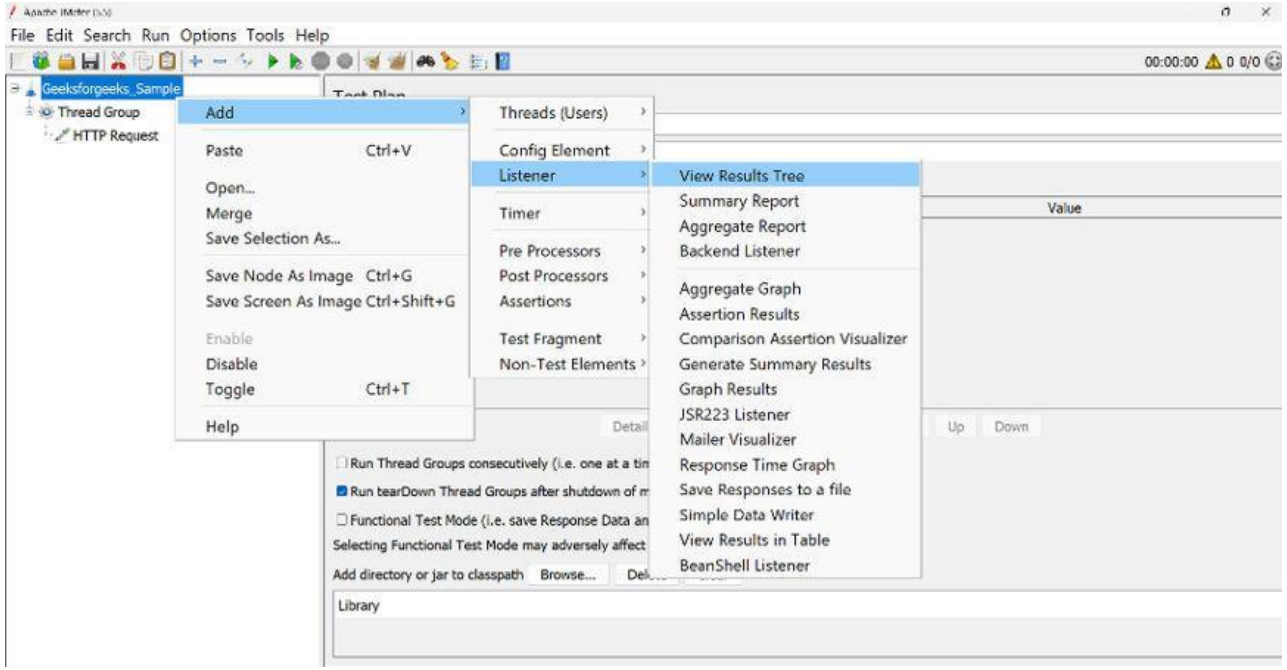



idat

# Guía paso a paso

## Paso 5: Agregar oyentes para los resultados

1. Haga clic derecho en **Grupo de subprocesos** y agregue un oyente (Agregar → Oyente → Ver árbol de resultados).
2. También puedes agregar otros oyentes como:
   - **Informe resumido:** proporciona métricas de rendimiento agregadas.
   - **Resultados gráficos:** muestra las tendencias de rendimiento de forma gráfica.
   - **Gráfico de tiempo de respuesta:** visualiza los tiempos de respuesta.




idat

# Guía paso a paso

## Paso 6: Configurar aserciones (opcional)

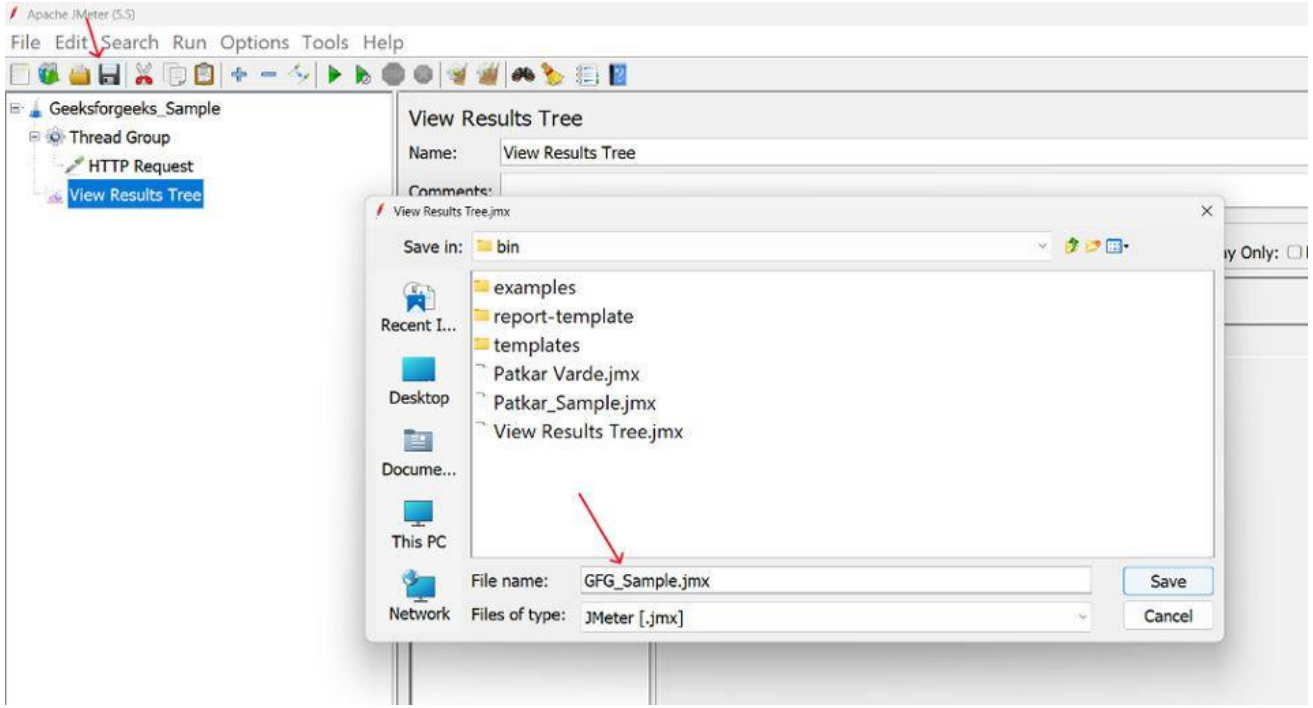
Las afirmaciones ayudan a verificar la exactitud de la respuesta. Para añadir una afirmación:

1. Haga clic derecho en la **solicitud HTTP** y agregue una afirmación (Agregar → Afirmaciones → Afirmación de respuesta).
2. Definir el texto esperado o códigos de respuesta (por ejemplo, 200 OK).

# Guía paso a paso

## Paso 7: Ejecutar la prueba

1. Haga clic en el botón **Iniciar** (icono de reproducción verde) para comenzar la prueba.
2. Supervisar la ejecución de pruebas en tiempo real en Listeners.
3. Analice los resultados utilizando informes y registros.

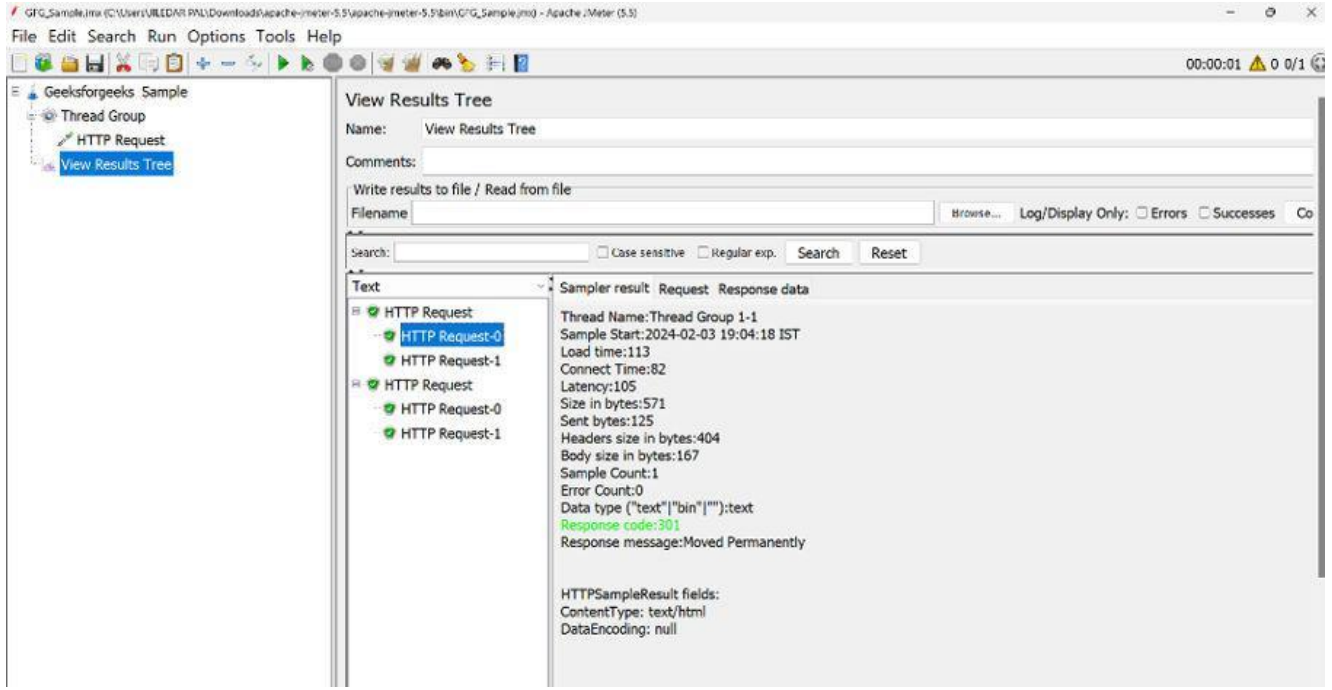



idat

# Guía paso a paso

## Paso 8: Analizar los resultados

- **Tiempo de respuesta:** tiempos de respuesta más bajos indican un mejor rendimiento.
- **Rendimiento:** valores más altos significan que el sistema puede manejar más solicitudes.
- **Porcentaje de error:** debe ser lo más bajo posible.
- **Uso de CPU y memoria:** ayuda a identificar cuellos de botella de hardware.




idat

# Guía paso a paso

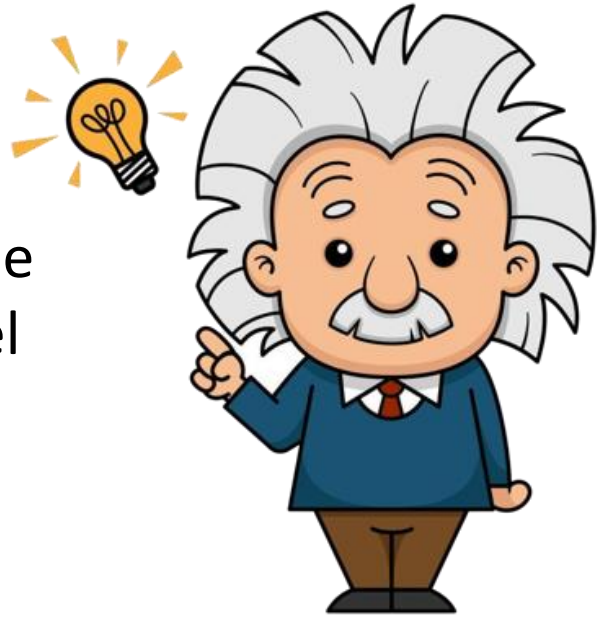
## Paso 9: Optimizar el rendimiento

Si surgen problemas de rendimiento, considere:

1. **Almacenamiento en caché:** implemente el almacenamiento en caché del lado del servidor para reducir el tiempo de procesamiento.
2. **Optimización de bases de datos:** optimice las consultas y la indexación.
3. **Equilibrio de carga:** distribuya la carga entre múltiples servidores.
4. **Redes de distribución de contenido (CDN):** mejore los tiempos de respuesta para usuarios globales.
5. **Escalabilidad:** actualice los recursos del servidor o adopte soluciones de escalabilidad en la nube.

# Guía paso a paso

JMeter es una herramienta multifacética con excelentes capacidades para realizar pruebas de rendimiento de aplicaciones y servicios web. El método de experimentación que se aplica consiste en realizar pruebas con la herramienta, garantizando así un rendimiento óptimo del tester, quien puede realizar escenarios reales como: estímulos de carga, análisis del funcionamiento del sistema e identificación de límites de rendimiento.



Son las pruebas de rendimiento que se realizan periódicamente las que mantienen las aplicaciones rápidas, confiables y escalables y, al mismo tiempo, aumentan la experiencia del usuario y el éxito empresarial.

idat

¿Dudas?

¿Miedos?

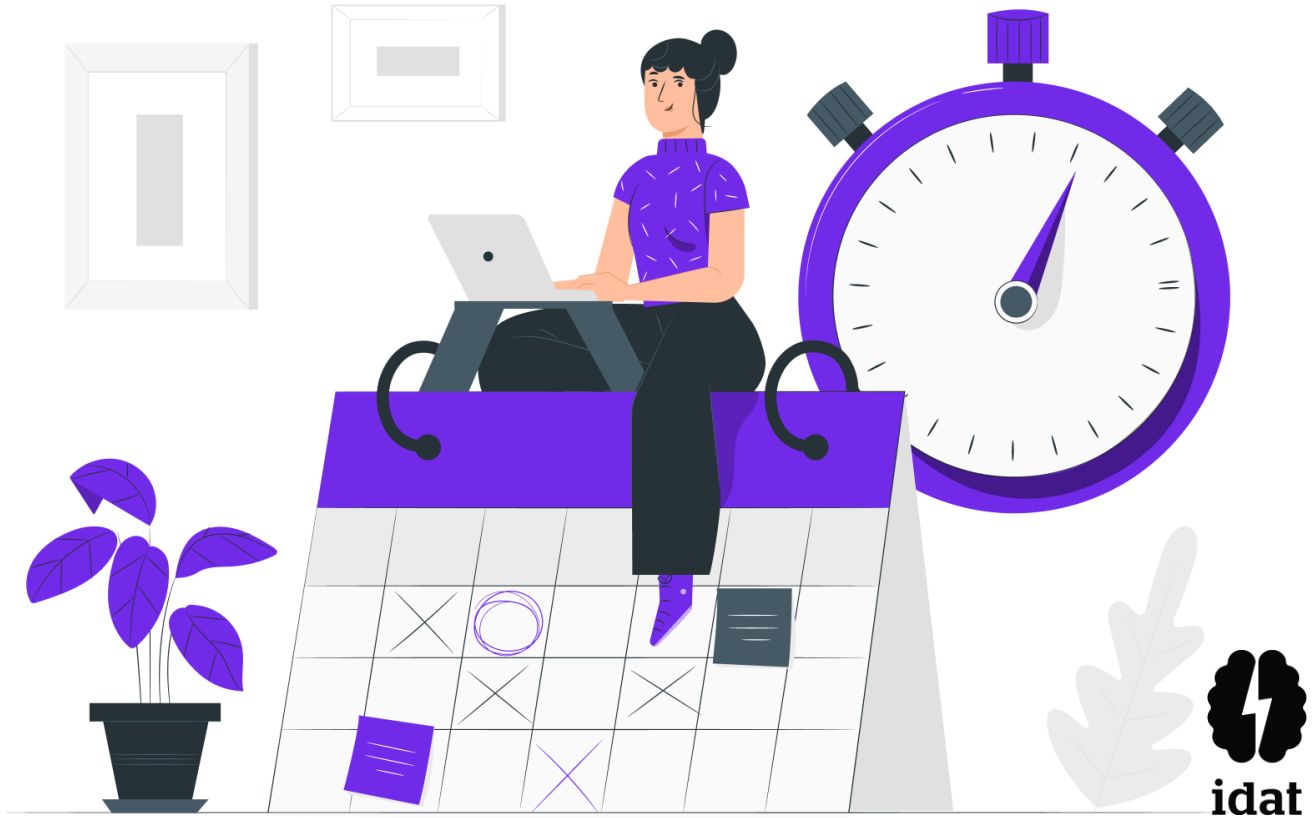
¿Temores?



idat

# Aplicamos lo aprendido: Asignación para la clase siguiente

Resolver la actividad en el EVA



idat



idat

# GRACIAS